Project 2: Natural Wildfire Ignition Frequency by Fuel Type Across CONUS Ecoregions, 1992–2024

Names: Jansen Rinck, Benjamin Barlog, Jianru Shen

Class: CSCI 502: Geospatial Data Science

Date: 10/12/2026

Introduction

Natural wildfire ignitions are not evenly distributed across the contiguous United States (CONUS), and the fuels present on the landscape are one factor shaping where they occur. By comparing ignition rates across fuel types and ecoregions, this analysis examines how fuels and fire interact, and how those interactions vary from region to region. The results can help fire and forest managers understand the spatial patterns of natural ignitions. This is increasingly important as ecosystems, especially post-fire landscapes with limited forest regeneration, shift away from historical vegetation structure. Analyses like this one can help assess the fire-regime shifts that may follow.

Research Question:

Between 2018 and 2020, does the rate at which lightning strikes convert into wildfire ignitions vary by fuel type across the ecoregions of the contiguous United States, independent of raw lightning exposure?

Revised Research Question: Between 1992 and 2024, how are natural wildfire ignitions distributed across fuel types in CONUS, which fuel types are most susceptible to natural ignitions relative to the area they cover, and how does each ecoregion's mix of fuels explain differences in ignition density between ecoregions?

Revised Hypothesis: Between 1992 and 2024, natural wildfire ignitions across CONUS are not distributed randomly - they are concentrated in specific fuel types. Timber-understory fuels, which combine forest litter with grass and shrubs in the understory, will be the most susceptible to natural ignitions relative to the area they cover. Differences in ignition density between ecoregions will be driven mainly by the mix of fuels each ecoregion contains.

Data:

1. Spatial wildfire occurrence data for the United States, 1992-2024 (7th Edition)(USFS)
   - Federal, State and Local fire data aggregated
   - Discovery date, final fire size, point location (1 sq mile accuracy), wildfire-cause
   - 2.7 million geo-referenced wildfire records
2. LANDFIRE's 40 Scott and Burgan Fire Behavior Fuel Model (FBFM40), CONUS raster (USFS & USGS)
   - 40 fuel models classifying surface fuel loading by type (grass, brush, timber, slash), size class, bed depth, and moisture of extinction
   - Fuel assignments aren't static - herbaceous fuels shift between live and dead through the growing season, and burned or otherwise disturbed areas are updated to reflect estimated recovery through 2022
3. Level III Ecoregions of the Conterminous United States (EPA)
   - Ecoregions mark areas of broadly similar ecosystems - a spatial framework for managing and comparing environmental resources across agencies
   - Delineated from patterns across multiple factors (geology, vegetation, climate, soils, land use, wildlife, hydrology), weighted differently region to region
   - Four Levels: Level I (15 regions, North America) → Level II (50) → Level III (85 in the conterminous US) → Level IV (further subdivisions)

Data for Further Analysis

4. Combined Geostationary Lightning Mapper (GLM) Hourly Thunder Hour Data Product (NASA)

- 2019 - 2025
- Two satellites GOES-18 (west) and GOES-19 (east)
- Thunder hour: an hour with detected lightning at a location - a frequency-based storm-activity metric, less biased by a sensor's geographic detection-sensitivity variation than raw flash counts
- Resolution: 5 km

Definitions: Scott and Burgan Fuel Models (FBFM40)

Scott and Burgan (2005) group their 40 fire behavior fuel models (plus 5 nonburnable models) by the fuel that primarily carries fire. Values in parentheses are the codes used in the LANDFIRE FBFM40 raster.

Group	Primary carrier of fire
NB – Nonburnable	No fuel load; fire will not spread
GR – Grass	Grass
GS – Grass-Shrub	Grass and shrubs combined
SH – Shrub	Shrub twigs, foliage, and shrub litter
TU – Timber-Understory	Forest litter with herbaceous or shrub fuels
TL – Timber Litter	Dead and down woody fuel
SB – Slash-Blowdown	Activity fuel or blowdown

Nonburnable (NB): There is no fuel load, so wildland fire will not spread.

Code (value)	Name	Description
NB1 (91)	Urban/Developed	Urban and suburban development that does not support wildland fire spread
NB2 (92)	Snow/Ice	Land covered by permanent snow or ice
NB3 (93)	Agricultural	Agricultural land maintained in a nonburnable condition (e.g., irrigated annual crops, mowed or tilled orchards)
NB8 (98)	Open Water	Open bodies of water such as lakes, rivers, and oceans
NB9 (99)	Bare Ground	Land without enough fuel to support fire spread (e.g., gravel pits, sparse arid desert, sand dunes, rock outcrops, beaches)

Grass (GR): The primary carrier of fire is grass, ranging from heavily grazed stubble or sparse natural grass to dense grass more than 6 feet tall. All GR models are dynamic: live herbaceous fuel shifts from live to dead as it dries.

Code (value)	Name	Description
GR1 (101)	Short, Sparse Dry Climate Grass	Short, sparse grass (naturally or from grazing) that may be discontinuous
GR2 (102)	Low Load, Dry Climate Grass	Grass with more load than GR1; fuelbed may be more continuous
GR3 (103)	Low Load, Very Coarse, Humid Climate Grass	Continuous, coarse, humid-climate grass; light load, about 2 ft deep

Code (value)	Name	Description
GR4 (104)	Moderate Load, Dry Climate Grass	Continuous dry-climate grass; more load and depth than GR2, about 2 ft deep
GR5 (105)	Low Load, Humid Climate Grass	Humid-climate grass; more load than GR3, 1–2 ft deep
GR6 (106)	Moderate Load, Humid Climate Grass	Continuous humid-climate grass; more load than GR5 and less coarse
GR7 (107)	High Load, Dry Climate Grass	Continuous dry-climate grass about 3 ft tall
GR8 (108)	High Load, Very Coarse, Humid Climate Grass	Continuous, very coarse humid-climate grass; spread and flame length can be extreme when cured
GR9 (109)	Very High Load, Humid Climate Grass	Dense humid-climate grass about 6 ft tall; spread and flame length can be extreme when cured

Grass-Shrub (GS): The primary carrier of fire is grass and shrubs combined; both determine fire behavior. All GS models are dynamic.

Code (value)	Name	Description
GS1 (121)	Low Load, Dry Climate Grass-Shrub	Shrubs about 1 ft high, low grass load; moderate spread, low flame length
GS2 (122)	Moderate Load, Dry Climate Grass-Shrub	Shrubs 1–3 ft high, moderate grass load; high spread, moderate flame length
GS3 (123)	Moderate Load, Humid Climate Grass-Shrub	Moderate grass/shrub load under 2 ft deep; high spread, moderate flame length
GS4 (124)	High Load, Humid Climate Grass-Shrub	Heavy grass/shrub load over 2 ft deep; high spread, very high flame length

Shrub (SH): The primary carrier of fire is live and dead shrub twigs and foliage together with dead and down shrub litter. SH1 and SH9 are dynamic.

Code (value)	Name	Description
SH1 (141)	Low Load Dry Climate Shrub	Low shrub load about 1 ft deep, some grass; very low spread and flame length
SH2 (142)	Moderate Load Dry Climate Shrub	Moderate load about 1 ft deep, no grass; low spread and flame length
SH3 (143)	Moderate Load, Humid Climate Shrub	Moderate shrub load, possibly under pine overstory, 2–3 ft deep; low spread and flame length
SH4 (144)	Low Load, Humid Climate Timber-Shrub	Low to moderate shrub and litter load, possibly under pine overstory, about 3 ft deep; high spread, moderate flame length
SH5 (145)	High Load, Dry Climate Shrub	Heavy shrub load 4–6 ft deep; very high spread and flame length

Code (value)	Name	Description
SH6 (146)	Low Load, Humid Climate Shrub	Dense shrubs, little or no herbaceous fuel, about 2 ft deep; high spread and flame length
SH7 (147)	Very High Load, Dry Climate Shrub	Very heavy shrub load 4–6 ft deep; high spread, very high flame length
SH8 (148)	High Load, Humid Climate Shrub	Dense shrubs, little or no herbaceous fuel, about 3 ft deep; high spread and flame length
SH9 (149)	Very High Load, Humid Climate Shrub	Dense, finely branched shrubs 4–6 ft tall with much fine dead fuel; high spread, very high flame length

Timber-Understory (TU): The primary carrier of fire is forest litter combined with herbaceous or shrub fuels. TU1 and TU3 are dynamic.

Code (value)	Name	Description
TU1 (161)	Low Load Dry Climate Timber-Grass-Shrub	Low load of grass and/or shrub with litter; low spread and flame length
TU2 (162)	Moderate Load, Humid Climate Timber-Shrub	Moderate litter load with a shrub component; moderate spread, low flame length
TU3 (163)	Moderate Load, Humid Climate Timber-Grass-Shrub	Moderate forest litter with grass and shrubs; high spread, moderate flame length
TU4 (164)	Dwarf Conifer With Understory	Short conifer trees with a grass or moss understory; moderate spread and flame length
TU5 (165)	Very High Load, Dry Climate Timber-Shrub	Heavy forest litter with a shrub or small-tree understory; moderate spread and flame length

Timber Litter (TL): The primary carrier of fire is dead and down woody fuel. Live fuel, if present, has little effect on fire behavior.

Code (value)	Name	Description
TL1 (181)	Low Load Compact Conifer Litter	Compact forest litter 1–2 in deep; can represent recently burned forest; very low spread and flame length
TL2 (182)	Low Load Broadleaf Litter	Low load, compact broadleaf (hardwood) litter; very low spread and flame length
TL3 (183)	Moderate Load Conifer Litter	Moderate conifer litter with a light load of coarse fuels; very low spread, low flame length
TL4 (184)	Small Downed Logs	Moderate fine litter and coarse fuels, including small-diameter downed logs; low spread and flame length

Code (value)	Name	Description
TL5 (185)	High Load Conifer Litter	High conifer litter load with light slash or mortality fuel; low spread and flame length
TL6 (186)	Moderate Load Broadleaf Litter	Moderate broadleaf litter, less compact than TL2; moderate spread, low flame length
TL7 (187)	Large Downed Logs	Heavy forest litter including larger-diameter downed logs; low spread and flame length
TL8 (188)	Long-Needle Litter	Moderate long-needle pine litter, possibly with some herbaceous fuel; moderate spread, low flame length
TL9 (189)	Very High Load Broadleaf Litter	Very high load, fluffy broadleaf litter (or heavy needle-drape); moderate spread and flame length

Slash-Blowdown (SB): The primary carrier of fire is activity fuel or blowdown. Forested areas with heavy mortality may also be modeled with SB.

Code (value)	Name	Description
SB1 (201)	Low Load Activity Fuel	Light dead and down activity fuel under 1 ft deep; moderate spread, low flame length
SB2 (202)	Moderate Load Activity Fuel or Low Load Blowdown	Moderate dead and down activity fuel or light blowdown, about 1 ft deep
SB3 (203)	High Load Activity Fuel or Moderate Load Blowdown	Heavy dead and down activity fuel or moderate blowdown, more than 1 ft deep
SB4 (204)	High Load Blowdown	Heavy, uncompacted blowdown with foliage and fine fuel still attached; very high spread and flame length

Source: Scott, J. H., & Burgan, R. E. (2005). Standard fire behavior fuel models: A comprehensive set for use with Rothermel's surface fire spread model (Gen. Tech. Rep. RMRS-GTR-153, pp. 19–70). USDA Forest Service, Rocky Mountain Research Station.

[1]:
```
# Import libraries

# Standard library
import gc
from pathlib import Path

# Data handling
import numpy as np
import pandas as pd
from IPython.display import display

# Geospatial
import geopandas as gpd
import rasterio
from rasterio.features import rasterize
from rasterio.transform import from_origin
```

```
from rasterio.windows import Window
from rasterio.windows import bounds as window_bounds
from shapely.geometry import box
from libpysal import weights

# Statistics / KDE
from scipy.ndimage import gaussian_filter
from scipy.stats import chi2
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.neighbors import KernelDensity

# Plotting
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap, LogNorm, Normalize, to_hex, to_rgb
from matplotlib.lines import Line2D
from matplotlib.patches import Patch, Polygon, Rectangle

# Interactive plotting (Bokeh: interactions run in the browser, so they also work in a saved HTML file)
from bokeh.io import output_notebook, save, show
from bokeh.layouts import column, row
from bokeh.models import (Checkbox, CheckboxButtonGroup, ColorBar, ColumnDataSource, CustomJS, Div,
                          FactorRange, HoverTool, LinearColorMapper, LogColorMapper,
 ↪NumeralTickFormatter,
                       RadioButtonGroup, Range1d, RangeSlider, Select, Slider, Span, TapTool, Whisker)
from bokeh.plotting import figure

DATA_DIR = Path("../data")
OUTPUT_DIR = Path("../outputs")
```

Reprojections for Analysis and Geographic Alignment

Projection: EPSG:5070 — NAD83 / Conus Albers Equal Area Conic

- Albers is an equal-area projection that preserves area accuracy for CONUS
- Minimal distortion for CONUS
- Well-suited for regional comparisons

[2]:
```
# Fire occurrence (7th edition, reprojected to Albers)
# Cache paths so we don't have to recompute every time we open the notebook
fire_cache_path = DATA_DIR / "processed/fire_occurrence_7th_albers.gpkg"
# The cache folder is gitignored, so a fresh clone doesn't have it yet
fire_cache_path.parent.mkdir(parents=True, exist_ok=True)

# Only load the columns the analysis uses (all 38 columns take ~4 GB of memory)
fire_columns = ["FIRE_YEAR", "DISCOVERY_DATE", "NWCG_CAUSE_CLASSIFICATION",
                "NWCG_GENERAL_CAUSE", "FIRE_SIZE", "STATE"]

if fire_cache_path.exists():
    fire = gpd.read_file(fire_cache_path, engine="pyogrio", columns=fire_columns)
else:
    # The GeoPackage also holds a reporting-unit table, so name the fire layer explicitly
    fire = gpd.read_file(DATA_DIR / "FPA_FOD/FPA_FOD_20260615.gpkg", layer="Fires",
                         engine="pyogrio", columns=fire_columns)
    fire = fire.to_crs("EPSG:5070")
    fire.to_file(fire_cache_path, driver="GPKG")

# Smaller dtypes: repeated strings become categories, numbers get narrower types
fire = fire.astype({"NWCG_CAUSE_CLASSIFICATION": "category",
                    "NWCG_GENERAL_CAUSE": "category", "STATE": "category",
```

```
                    "FIRE_YEAR": "int16", "FIRE_SIZE": "float32"})

# Fuel type raster (already in EPSG:5070 — no reprojection needed)
fuelClass = rasterio.open(DATA_DIR / "LF2020_FBFM40_CONUS/Tif/LF2020_FBFM40_CONUS.tif")

# Ecoregions (reprojected + dissolved to one polygon per ecoregion)
# Dissolve ecoregions because the downloaded shapefiles split ecoregion polygons at state-lines
ecoregion_cache_path = DATA_DIR / "processed/ecoregions_l3_albers_nb.gpkg"

if ecoregion_cache_path.exists():
    ecoregion = gpd.read_file(ecoregion_cache_path)
else:
    ecoregion = gpd.read_file(DATA_DIR / "ecoregions/us_eco_l3_state_boundaries.shp")
    ecoregion = ecoregion.to_crs("EPSG:5070")
    # Dissolve on the ecoregion identifier
    ecoregion = ecoregion.dissolve(by="US_L3CODE", as_index=False)
    ecoregion.to_file(ecoregion_cache_path, driver="GPKG")
```

Data Exploration and Cleaning

[3]:
```
# Look at the datasets to identify what needs to be cleaned
print("Fire Occurrence")
pd.set_option('display.max_columns', None)
display(fire.head(10))

print("Ecoregions")
display(ecoregion.head(10))

print("Fuel Model Classes (LANDFIRE attribute table)")
fuel_lookup = pd.read_csv(DATA_DIR / "LF2020_FBFM40_CONUS/CSV_Data/LF2020_FBFM40.csv")
display(fuel_lookup.head(10))
```

```
Fire Occurrence
```

	DISCOVERY_DATE	FIRE_SIZE	FIRE_YEAR	NWCG_CAUSE_CLASSIFICATION	NWCG_GENERAL_CAUSE	STATE	geometry
0	2005-02-02	0.10	2005	Human	Power generation/transmission/distribution	CA	
1	2004-05-12	0.25	2004	Natural	Natural	CA	
2	2004-05-31	0.10	2004	Human	Debris and open burning	CA	
3	2004-06-28	0.10	2004	Natural	Natural	CA	
4	2004-06-28	0.10	2004	Natural	Natural	CA	
5	2004-06-30	0.10	2004	Natural	Natural	CA	
6	2004-07-01	0.10	2004	Natural	Natural	CA	
7	2005-03-08	0.80	2005	Human	Debris and open burning	CA	
8	2005-03-15	1.00	2005	Human	Debris and open burning	CA	
9	2004-07-01	0.10	2004	Natural	Natural	CA	

```
0  POINT (-2090649.206 2167768.542)
1  POINT (-2072914.128 2035040.471)
2  POINT (-2098930.807 2047846.468)
3  POINT (-2042526.985 1983845.809)
4  POINT (-2044175.784 1984269.866)
5  POINT (-2056290.102 1996209.432)
6  POINT (-2058945.094 2003042.150)
7  POINT (-2178397.193 2300347.756)
8  POINT (-2158190.184 2325509.048)
9  POINT (-2062542.378 1987763.701)
```

Ecoregions

```
  US_L3CODE                US_L3NAME NA_L3CODE  \
0         1               Coast Range     7.1.8
1        10          Columbia Plateau    10.1.2
2        11            Blue Mountains     6.2.9
3        12         Snake River Plain    10.1.8
4        13  Central Basin and Range    10.1.5
5        14   Mojave Basin and Range    10.2.1
6        15          Northern Rockies     6.2.3
7        16           Idaho Batholith    6.2.15
8        17            Middle Rockies    6.2.10
9        18             Wyoming Basin    10.1.4

                            NA_L3NAME NA_L2CODE                  NA_L2NAME  \
0                          Coast Range       7.1  MARINE WEST COAST FOREST
1                       Columbia Plateau      10.1               COLD DESERTS
2                         Blue Mountains       6.2         WESTERN CORDILLERA
3                       Snake River Plain      10.1               COLD DESERTS
4                  Central Basin and Range      10.1               COLD DESERTS
5                   Mojave Basin and Range      10.2               WARM DESERTS
6  Columbia Mountains/Northern Rockies       6.2         WESTERN CORDILLERA
7                         Idaho Batholith       6.2         WESTERN CORDILLERA
8                          Middle Rockies       6.2         WESTERN CORDILLERA
9                           Wyoming Basin      10.1               COLD DESERTS

  NA_L1CODE                        NA_L1NAME  STATE_NAME  EPA_REGION  \
0         7         MARINE WEST COAST FOREST  California           9
1        10            NORTH AMERICAN DESERTS       Idaho          10
2         6  NORTHWESTERN FORESTED MOUNTAINS       Idaho          10
3        10            NORTH AMERICAN DESERTS       Idaho          10
4        10            NORTH AMERICAN DESERTS  California           9
5        10            NORTH AMERICAN DESERTS     Arizona           9
6         6  NORTHWESTERN FORESTED MOUNTAINS       Idaho          10
7         6  NORTHWESTERN FORESTED MOUNTAINS       Idaho          10
8         6  NORTHWESTERN FORESTED MOUNTAINS       Idaho          10
9        10            NORTH AMERICAN DESERTS    Colorado           8

                      L3_KEY                          L2_KEY  \
0               1  Coast Range  7.1  MARINE WEST COAST FOREST
1         10  Columbia Plateau             10.1  COLD DESERTS
2           11  Blue Mountains        6.2  WESTERN CORDILLERA
3        12  Snake River Plain             10.1  COLD DESERTS
4  13  Central Basin and Range             10.1  COLD DESERTS
5   14  Mojave Basin and Range             10.2  WARM DESERTS
6         15  Northern Rockies        6.2  WESTERN CORDILLERA
7          16  Idaho Batholith        6.2  WESTERN CORDILLERA
8           17  Middle Rockies        6.2  WESTERN CORDILLERA
9            18  Wyoming Basin             10.1  COLD DESERTS
```

```
                              L1_KEY  \
0         7  MARINE WEST COAST FOREST
1          10  NORTH AMERICAN DESERTS
2  6  NORTHWESTERN FORESTED MOUNTAINS
3          10  NORTH AMERICAN DESERTS
4          10  NORTH AMERICAN DESERTS
5          10  NORTH AMERICAN DESERTS
6  6  NORTHWESTERN FORESTED MOUNTAINS
7  6  NORTHWESTERN FORESTED MOUNTAINS
8  6  NORTHWESTERN FORESTED MOUNTAINS
9          10  NORTH AMERICAN DESERTS

                                             geometry
0  MULTIPOLYGON (((-2326956.693 1961322.753, -232…
1  MULTIPOLYGON (((-1528416.035 2731696.232, -152…
2  POLYGON ((-1672022.952 2558457.555, -1672093.0…
3  POLYGON ((-1202707.144 2395971.238, -1202710.6…
4  POLYGON ((-1858634.567 1762679.205, -1859348.8…
5  POLYGON ((-1579497.947 1659182.070, -1579385.6…
6  POLYGON ((-1423929.253 2779186.628, -1424035.9…
7  POLYGON ((-1423805.863 2779074.161, -1423720.2…
8  MULTIPOLYGON (((-1215843.351 2390715.034, -121…
9  POLYGON ((-948977.546 2056277.979, -948857.610…
```

Fuel Model Classes (LANDFIRE attribute table)

	VALUE	FBFM40	R	G	B	RED	GREEN	BLUE
0	-9999	Fill-NoData	255	255	255	1.000000	1.000000	1.000000
1	91	NB1	104	104	104	0.407843	0.407843	0.407843
2	92	NB2	225	225	225	0.882353	0.882353	0.882353
3	93	NB3	255	237	237	1.000000	0.929412	0.929412
4	98	NB8	0	14	214	0.000000	0.054902	0.839216
5	99	NB9	77	110	112	0.301961	0.431373	0.439216
6	101	GR1	255	235	190	1.000000	0.921569	0.745098
7	102	GR2	255	211	115	1.000000	0.827451	0.450980
8	103	GR3	255	236	139	1.000000	0.925490	0.545098
9	104	GR4	255	255	115	1.000000	1.000000	0.450980

[4]:
```
# Check each dataset for empty/NaN values
print(fire.isnull().sum())
print(ecoregion.isnull().sum())
print(fuel_lookup.isnull().sum())
```

```
DISCOVERY_DATE               0
FIRE_SIZE                    0
FIRE_YEAR                    0
NWCG_CAUSE_CLASSIFICATION    0
NWCG_GENERAL_CAUSE           0
STATE                        0
geometry                     0
dtype: int64
US_L3CODE     0
US_L3NAME     0
NA_L3CODE     0
NA_L3NAME     0
NA_L2CODE     0
NA_L2NAME     0
NA_L1CODE     0
NA_L1NAME     0
STATE_NAME    0
EPA_REGION    0
```

```
L3_KEY        0
L2_KEY        0
L1_KEY        0
geometry      0
dtype: int64
VALUE     0
FBFM40    0
R         0
G         0
B         0
RED       0
GREEN     0
BLUE      0
dtype: int64
```

[5]:
```
# Simplify the Fire Occurrence Dataset to suit the needs of the analysis
# Cleaned data is free of empty/NaN values
fire_clean = fire  # already limited to these columns at load

# Look at new dataframe
display(fire_clean.head(10))
```

```
  DISCOVERY_DATE  FIRE_SIZE  FIRE_YEAR NWCG_CAUSE_CLASSIFICATION  \
0     2005-02-02       0.10       2005                      Human
1     2004-05-12       0.25       2004                    Natural
2     2004-05-31       0.10       2004                      Human
3     2004-06-28       0.10       2004                    Natural
4     2004-06-28       0.10       2004                    Natural
5     2004-06-30       0.10       2004                    Natural
6     2004-07-01       0.10       2004                    Natural
7     2005-03-08       0.80       2005                      Human
8     2005-03-15       1.00       2005                      Human
9     2004-07-01       0.10       2004                    Natural

                          NWCG_GENERAL_CAUSE STATE  \
0  Power generation/transmission/distribution    CA
1                                     Natural    CA
2                   Debris and open burning    CA
3                                     Natural    CA
4                                     Natural    CA
5                                     Natural    CA
6                                     Natural    CA
7                   Debris and open burning    CA
8                   Debris and open burning    CA
9                                     Natural    CA

                         geometry
0  POINT (-2090649.206 2167768.542)
1  POINT (-2072914.128 2035040.471)
2  POINT (-2098930.807 2047846.468)
3  POINT (-2042526.985 1983845.809)
4  POINT (-2044175.784 1984269.866)
5  POINT (-2056290.102 1996209.432)
6  POINT (-2058945.094 2003042.150)
7  POINT (-2178397.193 2300347.756)
8  POINT (-2158190.184 2325509.048)
9  POINT (-2062542.378 1987763.701)
```

[6]:
```
natural_ignitions_cache_path = DATA_DIR / "processed/natural_fire_fuel_ecoregion.gpkg"

if natural_ignitions_cache_path.exists():
    # Ecoregion and fuel columns, plus year/date/size for the interactive summaries, plus the point
 ↪geometry for gridding
    natural_ignitions = gpd.read_file(natural_ignitions_cache_path, engine="pyogrio",
        columns=["US_L3NAME", "US_L3CODE", "FUEL_CLASS", "FIRE_YEAR", "DISCOVERY_DATE", "FIRE_SIZE"]
    ).astype({"FUEL_CLASS": "int16", "FIRE_YEAR": "int16", "FIRE_SIZE": "float32"})
# Only look at naturally caused fires
else:
    natural_fire = fire[fire["NWCG_CAUSE_CLASSIFICATION"] == "Natural"].copy()

    # Build a list of x,y pairs of point geom for natural fires
    coords = [(geom.x, geom.y) for geom in natural_fire.geometry]
    # Look up the rasters pixel by coords (val[0] removes the rasterio wrapper and returns an int to
 ↪the new col FUEL_CLASS to natural_fire
    natural_fire["FUEL_CLASS"] = [val[0] for val in fuelClass.sample(coords)]

    # Spatial join tags each fire WITHIN its containing ecoregion
    # Geopandas spatial join matches rows based on geom
    natural_ignitions = gpd.sjoin(natural_fire, ecoregion, how="left", predicate="within")

    natural_ignitions.to_file(natural_ignitions_cache_path, driver="GPKG")
    del natural_fire, coords

# Clip fire points to the lower 48
# The fire data covers all of the US, but the ecoregions only cover the lower 48.
# The left join in the natural_ignitions cell kept every fire, so fires outside the
# ecoregions have NaN in the ecoregion columns. Keeping only rows with an ecoregion name.
natural_ignitions = natural_ignitions[natural_ignitions["US_L3NAME"].notna()].copy()

# Raw ignition counts per ecoregion / fuel type combination
counts = natural_ignitions.groupby(["US_L3NAME", "FUEL_CLASS"]).size().
 ↪reset_index(name="ignition_count")
print(counts)

# Free the full fire table now that natural_ignitions is built
del fire, fire_clean
gc.collect()
```

```
                    US_L3NAME  FUEL_CLASS  ignition_count
0     Acadian Plains and Hills          91              37
1     Acadian Plains and Hills          98              12
2     Acadian Plains and Hills         101               4
3     Acadian Plains and Hills         102              19
4     Acadian Plains and Hills         103               8
…                          …           …               …
1730            Wyoming Basin         183             142
1731            Wyoming Basin         184               5
1732            Wyoming Basin         185               2
1733            Wyoming Basin         186              42
1734            Wyoming Basin         188              39

[1735 rows x 3 columns]
```

[6]:
```
0
```

[7]:
```
# Remove fires that fall in NB1 Urban/developed and the unburnable fuel types
# NB9 Barren, NB8 Open water, & NB2 Snow/ice, -9999 Fill-NoData
```

```
# Also remove SB1-SB4 Slash-blowdown: it covers only ~300 km² of CONUS (24 fires), too little data to
 ↪analyze

# look up each fuel class that is unburnable (plus urban/developed and slash-blowdown)
is_unburnable = (fuel_lookup["FBFM40"].isin(["NB1", "NB2", "NB8", "NB9", "Fill-NoData"])
                 | fuel_lookup["FBFM40"].str.startswith("SB"))
# Assign the values of the unburnable fuel classes to unburnable
unburnable = fuel_lookup.loc[is_unburnable, "VALUE"]

# Drop unburnable fuel classes in the natural_ignitions df
dropped = natural_ignitions["FUEL_CLASS"].isin(unburnable)

# How many fires were dropped?
print(f"Dropping {dropped.sum():,} of {len(natural_ignitions):,} fires ({dropped.mean():.1%}) "
      "on urban, snow/ice, water, barren, slash-blowdown, or no-data fuel")
print(natural_ignitions.loc[dropped, "FUEL_CLASS"].value_counts())

# This keeps the fires that were not flagged to be dropped and flips
# every true to false and false to true so true means keep (~ means not)
natural_ignitions = natural_ignitions[~dropped].copy()
```

```
Dropping 31,273 of 350,421 fires (8.9%) on urban, snow/ice, water, barren,
slash-blowdown, or no-data fuel
FUEL_CLASS
91     24286
99      5297
98      1653
202       18
92        13
201        6
Name: count, dtype: int64
```

Fuel availability: area of each fuel type in each ecoregion

Ignition counts favor fuels that cover a lot of ground. To identify which fuels are most receptive to ignition, area is calculated for each fuel class inside each ecoregion to create a normalized scale. Then ignitions are compared to that area (ignitions per km² of fuel).

The 30 m fuel raster has far too many pixels to load at once so we will batch process them, For each batch:

1. read the fuel codes
2. turn the ecoregion polygons into a matching array, so every pixel has a tag that corresponds to its ecoregion
3. count every (ecoregion, fuel class) pair and add it to a running total

[8]:
```
# Before we normalize we need to count the number of fuel-type pixels that fall within each ecoregion.
 ↪ This is what
# we will do below

# Lookup from fuel code to its FBFM40 label (e.g. 102 -> "GR2"), used to group fuel classes
code_to_label = dict(zip(fuel_lookup["VALUE"], fuel_lookup["FBFM40"]))

fuel_area_cache_path = DATA_DIR / "processed/fuel_area_by_ecoregion.csv"

if fuel_area_cache_path.exists():
    fuel_area = pd.read_csv(fuel_area_cache_path)
else:
    window_size = 4096
```

```
    # Ecoregion codes are stored as text and rasterize needs numbers, so convert text to ints
    eco_codes = ecoregion["US_L3CODE"].astype(int)

    # Create a running totals array that has space for each ecoregion/fuel pair
    pair_totals = np.zeros(256 * 256, dtype="int64")

    # Create a double for-loop to step through the fuel-class raster one window (4096 x 4096 pixels)
↪at a time
    for row_off in range(0, fuelClass.height, window_size):
        for col_off in range(0, fuelClass.width, window_size):
            # Window(col_off, row_off, width, height); min() trims the last windows at the right/
↪bottom edges
            window = Window(col_off, row_off,
                            min(window_size, fuelClass.width - col_off),
                            min(window_size, fuelClass.height - row_off))

            # Window_bounds converts the pixels into geographic coordinates in meters
            window_box = box(*window_bounds(window, fuelClass.transform))
            # Which ecoregions overlap this window? Returns T/F for each ecoregion
            overlapping = ecoregion.geometry.intersects(window_box)
            if not overlapping.any():
               continue   # window is entirely ocean / Canada / Mexico: nothing to count, skip the read

            # 1. Fuel codes for this window (int16 array, -9999 = no data)
            fuel = fuelClass.read(1, window=window)

            # 2. Ecoregion code for each pixel in the same window (0 = outside every ecoregion)
            #  Converts polygons into a grid of pixels. Input = geom and value, return = array where
↪every pixel falling
            # within that shape is assigned the shape's value
            eco = rasterize(zip(ecoregion.geometry[overlapping], eco_codes[overlapping]),
                            out_shape=fuel.shape,
                            transform=fuelClass.window_transform(window),
                            fill=0, dtype="uint8")

            # 3. Keep pixels that are inside an ecoregion and have fuel data, then count each pair
            valid = (eco > 0) & (fuel != -9999)
            keys = eco[valid].astype("int32") * 256 + fuel[valid]
            pair_totals += np.bincount(keys, minlength=pair_totals.size)   # bincount = how many
↪times each key appears

    # Turn the non-zero slots back into a table: key // 256 = ecoregion, key % 256 = fuel class
    keys = np.nonzero(pair_totals)[0]
    fuel_area = pd.DataFrame({"US_L3CODE": keys // 256,
                              "FUEL_CLASS": keys % 256,
                              "pixels": pair_totals[keys]})
    fuel_area.to_csv(fuel_area_cache_path, index=False)   # slow step runs once, then it's cached

# Each pixel is 30 m x 30 m = 900 m² = 0.0009 km²
fuel_area["area_km2"] = fuel_area["pixels"] * 900 / 1e6

# Same fuel grouping as the fires: two-letter FBFM40 prefix (fuel-class group)
# other non-burnable classes (urban, snow/ice, water, barren) get no group
fuel_labels = fuel_area["FUEL_CLASS"].map(code_to_label)
fuel_area["FUEL_GROUP"] = fuel_labels.str[:2].replace({"NB": None})
fuel_area.loc[fuel_labels == "NB3", "FUEL_GROUP"] = "Ag"

# Sanity check: the total should be close to the area of CONUS minus the Great Lakes, which the
# ecoregions leave out (~8.1 million km² - ~0.24 million km² ≈ 7.8 million km²)
```

```
print(f"Total area counted: {fuel_area['area_km2'].sum():,.0f} km²")
print(fuel_area.groupby("FUEL_GROUP")["area_km2"].sum().sort_values(ascending=False).round(0))
```

```
Total area counted: 7,789,736 km²
FUEL_GROUP
GR    2224221.0
TL    1431520.0
Ag    1168189.0
GS    1043392.0
TU     673194.0
SH     525452.0
SB        288.0
Name: area_km2, dtype: float64
```

Data Cleaning Discussion:

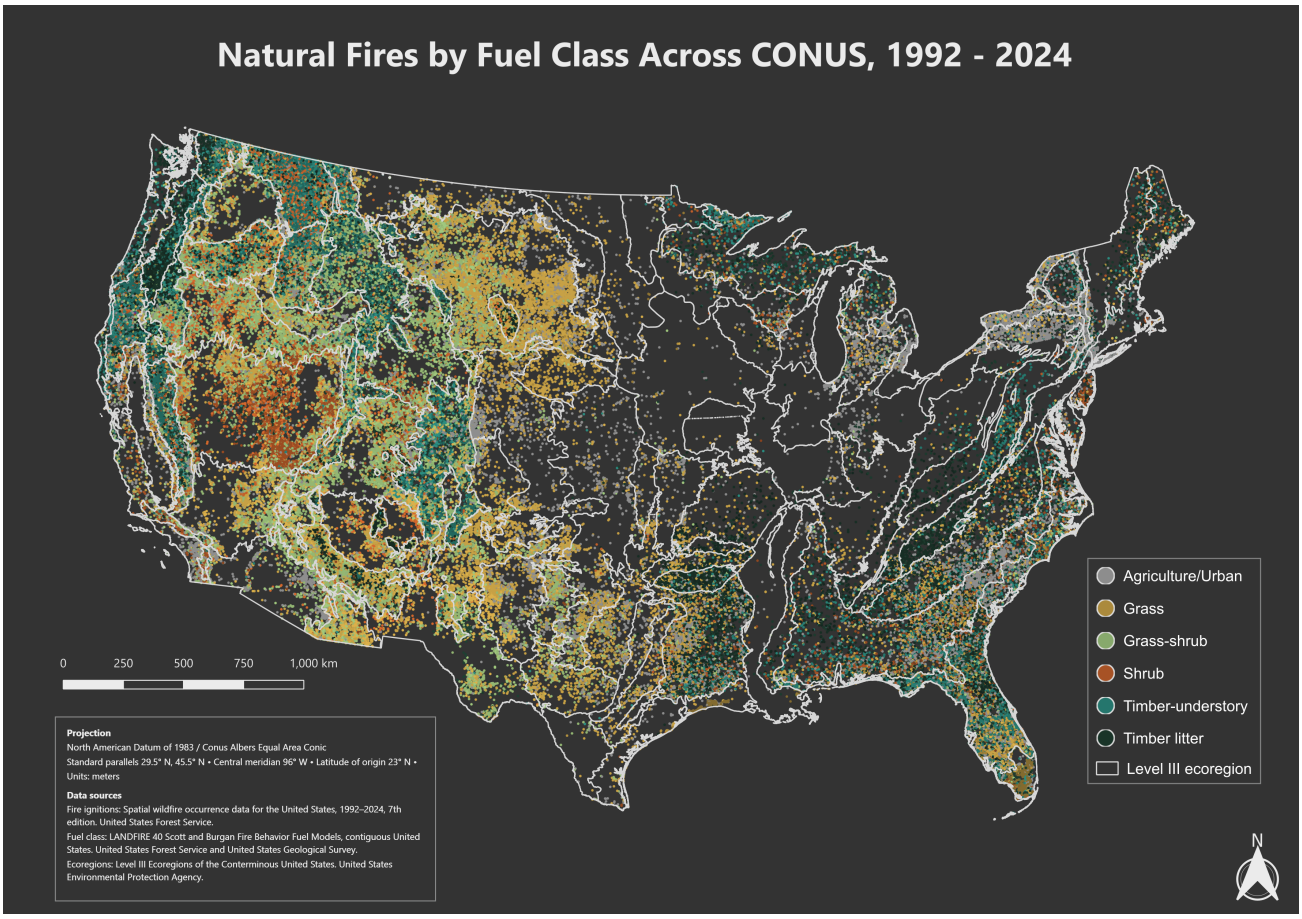
Fire Occurrence Data

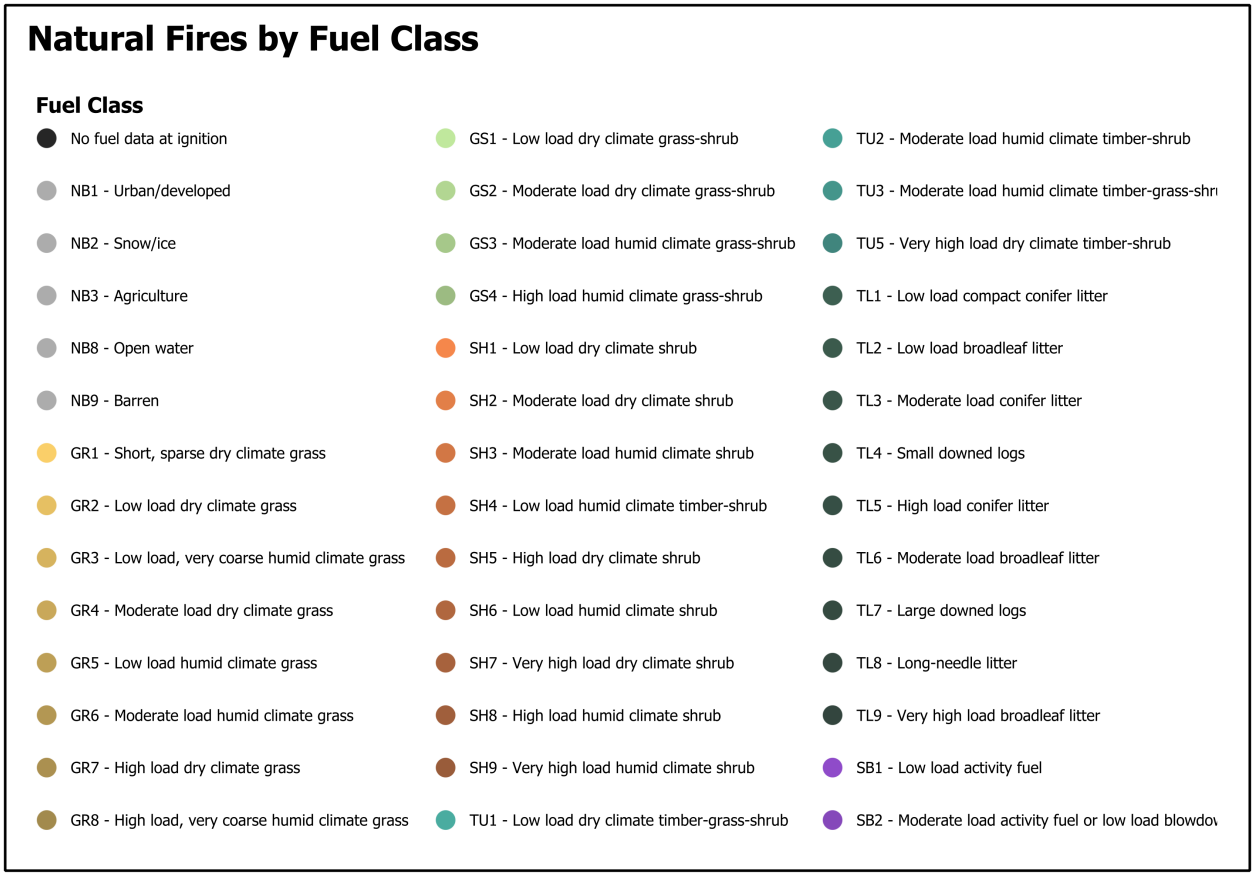
- Filtered to NWCG_Cause_Classification == “Natural”
- Clipped to CONUS through spatial join (point-in-polygon)
- Fire tagged with fuel class (30 m pixel) at its location
- Removed fires that were tagged with urban (NB1), snow/ice (NB2), water (NB8), barren (NB9), slash-blowdown (SB1–SB4) or no-data fuel

Ecoregions (EPA Level III)

- Dissolved on the US_L3CODE field because original dataset split polygons at state lines

Overview: Natural Ignitions by Fuel Class





View 1. Natural-cause wildfire ignitions across CONUS, before non-burnable and slash-blowdown fires were removed. Each point is one ignition, colored by the LANDFIRE FBFM40 fuel class at its location.

Discussion – Overview

This preliminary visualization shows only broad trends in ignitions by fuel type across CONUS. The sheer number of points makes it difficult to see patterns finer than the landscape scale. As expected, ignitions are concentrated in the West, specifically the Great Basin, the Northern and Middle Rockies, the Colorado Plateau, and the Arizona/New Mexico mountains, and they taper off toward the Midwest. Interestingly, Florida stands out as a hotspot in the East, possibly because of frequent lightning producing many small fires. Fuel classes at ignition points change clearly across ecoregions and often line up with ecoregion boundaries, which supports comparing ignitions by ecoregion. However, raw ignition counts can make a fuel type look dominant simply because it covers more area than other fuels, which shows the need to normalize ignition rates by fuel area. Finally, some ignitions fall on non-burnable fuel types, likely reflecting the ~1 mi² location accuracy of the fire records. Those records were removed during data cleaning.

Iteration 1: Kernel Density of Natural Ignitions

[9]:
```
# ---- Map elements used on the maps: scale bar, north arrow, projection / data sources box ----
# All positions are in map units (meters, EPSG:5070) unless noted.

def add_scalebar(ax, x0, y0, length_km=500, n_segments=4, height_km=30,
                 label_every_segment=False, labels_above=False, fill=("black", "white"),
 ↪color="black"):
    """Alternating black/white scale bar, lower-left corner at (x0, y0).
    label_every_segment=True labels every segment boundary (0, 250, 500, ...) like the overview map;
    otherwise only the start, middle and end. labels_above=True puts the labels above the bar.
    fill: the two alternating segment colors; color: outline and label color (for dark maps)."""
    seg = length_km * 1000 / n_segments                    # one segment's length in meters
```

```
    for i in range(n_segments):
        ax.add_patch(Rectangle((x0 + i * seg, y0), seg, height_km * 1000,
                               facecolor=fill[i % 2],                          # alternate colors
                               edgecolor=color, linewidth=0.8, zorder=5))
    # Where the labels go: every segment boundary, or start / middle / end
    if label_every_segment:
        ticks = [i * length_km / n_segments for i in range(n_segments + 1)]
    else:
        ticks = [0, length_km / 2, length_km]
    # Labels sit just above or just below the bar
    if labels_above:
        label_y, va = y0 + height_km * 1000 * 1.5, "bottom"
    else:
        label_y, va = y0 - height_km * 1000 * 0.5, "top"
    for km in ticks:
        ax.text(x0 + km * 1000, label_y, f"{km:,g}", ha="center", va=va, fontsize=9, color=color,
↪zorder=5)
    # "km" right after the end number, on the same line. It's offset in points (not meters) from the
    # bar's end: about half the width of the end number + a small gap, so the number stays centered
↪on its tick
    end_label = f"{length_km:,g}"
    ax.annotate("km", xy=(x0 + length_km * 1000, label_y),
                xytext=(0.3 * 9 * len(end_label) + 2.5, 0), textcoords="offset points",
                ha="left", va=va, fontsize=9, color=color, zorder=5)

def add_compass(ax, cx, cy, radius_km=90, style="arrow", fg="black", bg="white"):
    """North indicator centered at (cx, cy) in map units.
    style="arrow":   simple arrowhead, half black / half white, with N above it
    style="compass": 4-point compass rose, each point half black / half white
    fg / bg replace black / white (fg also colors the outline and the N), e.g. for dark maps.
    """
    r = radius_km * 1000          # length of each point
    w = r * 0.22                  # half-width of each point at its base
    if style == "arrow":
        # One tall arrowhead: tip at the top, notched base so it reads as an arrow, not a triangle
        tip, base_y, notch = (cx, cy + r), cy - r, (cx, cy - r * 0.55)
        w = r * 0.4
        ax.add_patch(Polygon([tip, (cx - w, base_y), notch], facecolor=fg, edgecolor=fg, lw=0.8,
↪zorder=5))
        ax.add_patch(Polygon([tip, (cx + w, base_y), notch], facecolor=bg, edgecolor=fg, lw=0.8,
↪zorder=5))
    else:
        # Each point = two triangles from the center: (tip, side) gives one half of the point
        for dx, dy in [(0, 1), (1, 0), (0, -1), (-1, 0)]:          # N, E, S, W
            tip = (cx + dx * r, cy + dy * r)
            left = (cx - dy * w, cy + dx * w)                      # perpendicular offsets at the base
            right = (cx + dy * w, cy - dx * w)
            ax.add_patch(Polygon([(cx, cy), left, tip], facecolor=fg, edgecolor=fg, lw=0.8, zorder=5))
            ax.add_patch(Polygon([(cx, cy), right, tip], facecolor=bg, edgecolor=fg, lw=0.8, zorder=5))
    ax.text(cx, cy + r * 1.1, "N", ha="center", va="bottom", fontsize=13, fontweight="bold",
↪color=fg, zorder=5)

def add_projection_box(ax, sources=None, x=0.0, y=-0.02, color="black", facecolor="white"):
    """Projection (and optional data sources) box, same wording as the overview map.
    x, y are in axes fractions (0-1); the default hangs the box just below the map's lower-left corner.
    sources: list of "Label: citation" strings; leave out to show only the projection.
    color: text color; facecolor: box fill (for dark maps)."""
    lines = [r"$\bf{Projection}$",
             "North American Datum of 1983 / Conus Albers Equal Area Conic",
```

```
            "Standard parallels 29.5° N, 45.5° N • Central meridian 96° W • Latitude of origin 23° N
 ↪•",
            "Units: meters"]
    if sources:
        lines += ["", r"$\bf{Data\ sources}$"] + sources
    return ax.text(x, y, "\n".join(lines), transform=ax.transAxes, ha="left", va="top", fontsize=8,
                   color=color, linespacing=1.4,
                   bbox=dict(facecolor=facecolor, edgecolor="0.5", boxstyle="square,pad=0.8"))
```

[10]:
```
# Cross-validate KDE bandwidth (σ)
# For each candidate σ, fit a Gaussian KDE on 4/5 of the fires and score how well it predicts
# too small overfits individual fires, too large smooths away patterns.
# Uses all fires because the best σ shrinks as the number of points grows.

# Pull out the natural_ignitions coordinates in meters. The KDE needs projected coordinates
 ↪(meters), not lat/lon
fire_xy = np.column_stack([natural_ignitions.geometry.x, natural_ignitions.geometry.y])   # EPSG:
 ↪5070 meters

# Create a cross-validation bandwidth function that uses grid search to find the optimal sigma
def cv_bandwidth(candidates):
    search = GridSearchCV(
        KernelDensity(kernel="gaussian", rtol=1e-4),
        {"bandwidth": candidates},                      # σ values to try, in meters
        # 5 folds, shuffled: the file is ordered by agency/region, so shuffling keeps
        # the folds from being grouped by region
        cv=KFold(n_splits=5, shuffle=True, random_state=0),
        n_jobs=-1,                                      # use all CPU cores
    )
    search.fit(fire_xy)
    return search.best_params_["bandwidth"], search.cv_results_["mean_test_score"]

# 2-stage coarse search over 2-30 km, then a finer search around the optimal sigma
# Makes 12 bandwidths from 2-30km
coarse = np.geomspace(2_000, 30_000, 12)
# Runs the cross-validation on all 12 values and returns the best σ and the average score for each one
best, scores = cv_bandwidth(coarse)
# Returns the position of the highest score
i = int(np.argmax(scores))
# Use i for the fine search to make 7 candidates evenly spaced around the optimal sigma's two neighbors
fine = np.linspace(coarse[max(i - 1, 0)], coarse[min(i + 1, len(coarse) - 1)], 7)
# Return the best σ from the fine cv
best_bandwidth, _ = cv_bandwidth(fine)

print(f"Cross-validated bandwidth: {best_bandwidth / 1000:.1f} km")
```

```
Cross-validated bandwidth: 5.1 km
```

[11]:
```
# Step 1: 1 km grid over CONUS
# Fires are counted per cell on this grid, then smoothed into density surfaces (KDE).
# 1 km keeps local variation in ignitions; FPA FOD locations are only accurate to ~1 sq mi,
# so going finer would add memory without adding real detail.

cell_size = 1_000  # grid cell width/height in meters (EPSG:5070 units)

# Grid extent = bounding box of the ecoregions (i.e., CONUS)
xmin, ymin, xmax, ymax = ecoregion.total_bounds

# Columns/rows needed to cover that box; ceil so the far edges aren't cut off
```

```
ncols = int(np.ceil((xmax - xmin) / cell_size))
nrows = int(np.ceil((ymax - ymin) / cell_size))

# Affine transform: ties grid row/column positions to real-world coordinates.
grid_transform = from_origin(xmin, ymax, cell_size, cell_size)

# CONUS mask: True for cells inside an ecoregion, False for ocean/Canada/Mexico
# all_touched=True keeps coastal/border cells that are only partly inside, so no fires get masked out
conus_mask = rasterize(ecoregion.geometry, out_shape=(nrows, ncols), transform=grid_transform,
                       fill=0, default_value=1, dtype="uint8", all_touched=True).astype(bool)

# Each fire's grid cell: x counts columns right from xmin, y counts rows down from ymax
natural_ignitions["COL"] = ((natural_ignitions.geometry.x - xmin) // cell_size).astype("int32")
natural_ignitions["ROW"] = ((ymax - natural_ignitions.geometry.y) // cell_size).astype("int32")

print(f"Grid: {nrows:,} rows x {ncols:,} cols = {nrows * ncols / 1e6:.1f}M cells "
      f"({conus_mask.sum() / 1e6:.1f}M inside CONUS)")

# Step 2: Count fires per 1 km cell
# Each fire adds 1 to the cell it falls in (np.add.at handles many fires in the same cell)
fire_counts = np.zeros((nrows, ncols), dtype="int32")
np.add.at(fire_counts, (natural_ignitions["ROW"].to_numpy(), natural_ignitions["COL"].to_numpy()), 1)

print(f"Total fires counted: {fire_counts.sum():,} (should equal {len(natural_ignitions):,})")
print(f"Cells with at least one fire: {(fire_counts > 0).sum():,}")
print(f"Most fires in one cell: {fire_counts.max()}")


# Step 3: Gaussian KDE of natural ignitions on the 1 km grid
bandwidth = round(best_bandwidth, -2)  # kernel standard deviation (σ) in meters, from cross-
validation
sigma_cells = bandwidth / cell_size    # gaussian_filter works in cells, not meters -> 5.1 cells
                                       # in 2-D, about 39% of each fire's weight stays within 1σ (~5
 ↪km) of it and about 86% within 2σ (~10 km)

density = gaussian_filter(
    fire_counts.astype("float32"),     # input grid (must be float)
    sigma=sigma_cells,                 # width of the Gaussian, in cells
    mode="constant",                   # treat everything beyond the grid edge as zero fires
    cval=0.0,
    truncate=4.0,                      # ignore cells farther than 4σ
)

# Each cell is 1 km², so the value is already fires per km² (1992-2024)
density[~conus_mask] = np.nan          # blank out ocean / Canada / Mexico (~ = not in)

print(f"Total before: {fire_counts.sum():,}  after: {np.nansum(density):,.0f}")

# Create a plot
fig, ax = plt.subplots(figsize=(12, 8))
im = ax.imshow(density, extent=(xmin, xmin + ncols * cell_size, ymax - nrows * cell_size, ymax),
               cmap="YlOrRd", norm=LogNorm(vmin=1e-3, vmax=np.nanmax(density)))
ecoregion.boundary.plot(ax=ax, color="0.3", linewidth=0.3)
cbar = fig.colorbar(im, ax=ax, shrink=0.6, label="Natural ignitions per km² (1992-2024)")
ax.set_title(f"Kernel Density of Natural Ignitions, 1992-2024 (Gaussian, σ = {bandwidth / 1000:.1f}
 ↪km)", fontsize=16)

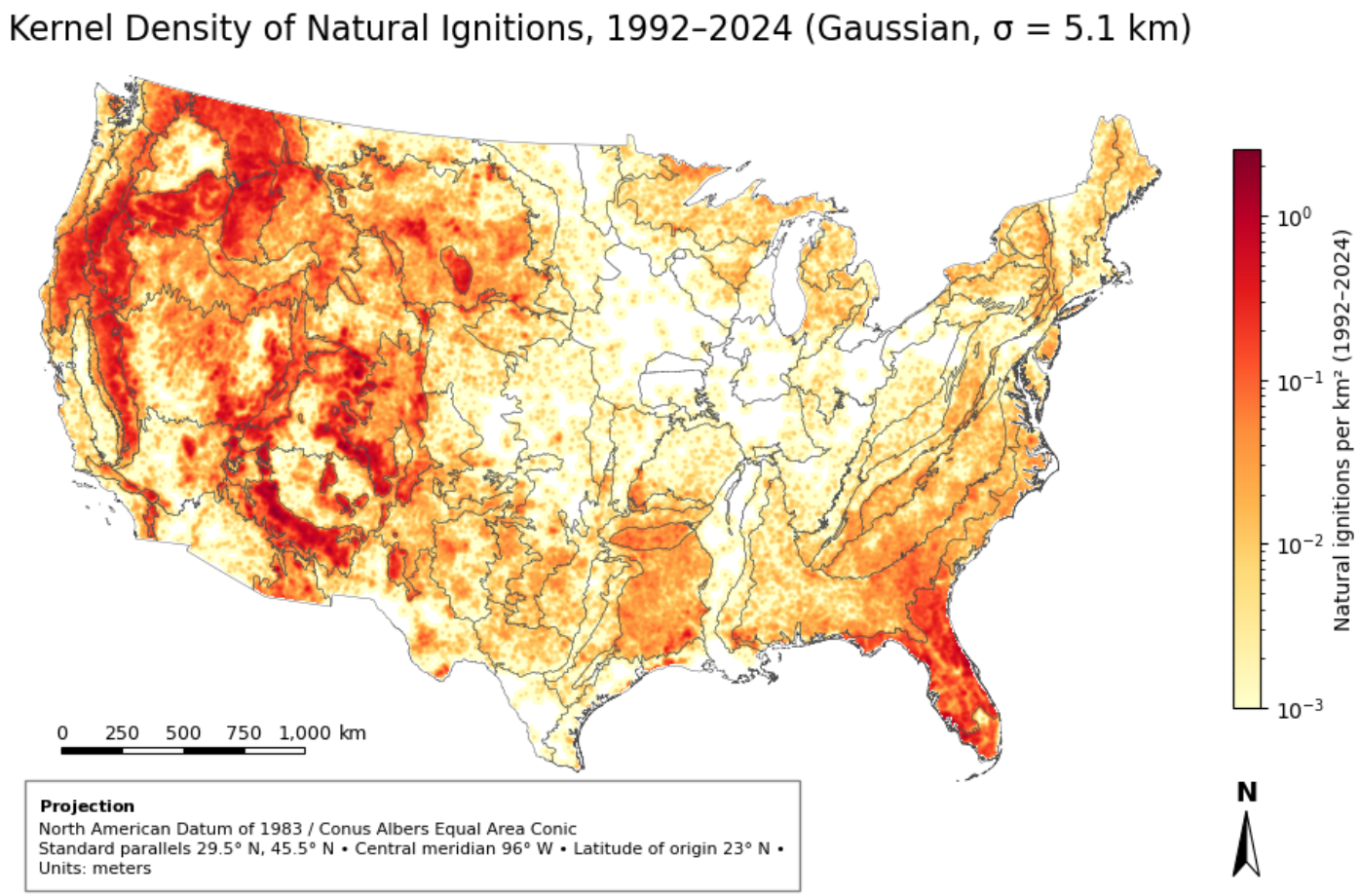
# Map elements laid out like the overview map: 1,000 km scale bar (labels above) in the lower left
# with the projection box below it, north arrow below the colorbar in the bottom-right corner
```

```
x_lo, x_hi = ax.get_xlim()
y_lo, y_hi = ax.get_ylim()
add_scalebar(ax, x_lo + 0.02 * (x_hi - x_lo), y_lo + 0.04 * (y_hi - y_lo), length_km=1000,
             n_segments=4, height_km=25, label_every_segment=True, labels_above=True)
add_projection_box(ax)

# North arrow: its own small axes centered under the colorbar (positions in figure fractions)
fig.canvas.draw()                       # settle the layout so the colorbar's final position is known
cb = cbar.ax.get_position()
arrow_ax = fig.add_axes([cb.x0 + cb.width / 2 - 0.03, cb.y0 - 0.16, 0.06, 0.11])
arrow_ax.set_xlim(-150_000, 150_000)
arrow_ax.set_ylim(-100_000, 150_000)
arrow_ax.set_aspect("equal")
add_compass(arrow_ax, 0, 0)
arrow_ax.set_axis_off()

ax.set_axis_off()
plt.show()
```

```
Grid: 2,901 rows x 4,615 cols = 13.4M cells (7.8M inside CONUS)
Total fires counted: 319,148 (should equal 319,148)
Cells with at least one fire: 255,235
Most fires in one cell: 18
Total before: 319,148  after: 318,051
```



View 2. Kernel density of natural-cause wildfire ignitions across CONUS. Colors show natural ignitions per km² on a log scale. Grey lines are EPA Level III ecoregion boundaries.

Discussion – First Iteration

Wildfire ignitions are heavily concentrated, with clear boundaries in particular ecoregions in the West and Southeast. About half of the 319,148 natural ignitions (~159,500) fall within just 401,247 km², or 5.1% of CONUS, and 80% (~255,300) fall within 1,237,645 km², or 15.8% of CONUS. The highest mean density of ignitions is in the Southwest, in the Arizona/New Mexico Mountains ecoregion (Upper Gila Mountains Level II ecoregion), with nearly 0.3 ignitions/km². High-density bands closely follow ecoregion boundaries in the Cascades, Sierra Nevada, Blue Mountains, Idaho Batholith, and Northern Rockies - all five lie within the same Western Cordillera Level II ecoregion. These regions share forest fuel types in elevated terrain that sees frequent summer thunderstorms. Also noteworthy, Florida is an outlier compared to its East Coast neighbors, and a clear hotspot exists in the Black Hills of South Dakota (Middle Rockies ecoregion). Raw density doesn't tell us enough about fuel groups to draw any definitive conclusions - although we can see that many of the western hotspots are in areas dominated by the timber understory and timber litter fuel groups.

Data Processing & Visualization

Kernel Density & Rasterizing

Creating a rasterized map of natural wildfire ignitions from point features first requires smoothing the data and taking it from discrete to continuous data. Choosing a function and the appropriate bandwidth to use in the KDE are the two most important design features when implementing a KDE. A Gaussian curve is a function that generally works well with a variety of data. The bandwidth to choose for the Gaussian is less straightforward and requires its own analysis. Using grid search cross-validation to find the optimal bandwidth is one approach to optimizing this value. For each candidate bandwidth (σ), cross-validation splits the fires into 5 random groups (folds) and fits a Gaussian KDE on 4 of the 5 groups, holding out the 5th for validation. It scores the bandwidth on the held-out fold by comparing the density of the KDE to where the validation group's fires actually fall, using log-likelihood. Higher scores correspond to KDEs that capture the distribution more accurately. This process repeats 5 times so each group is held out once, then the log-likelihood scores are averaged. This approach penalizes bandwidths that are too small because the Gaussian is a narrow spike over each fire occurrence - held-out fires that fall outside the spikes are given a density of ~0. The log of a density close to 0 is a large negative value - a few fires in the gaps between the curves drag the average score way down. This results in the KDE memorizing the training data and overfitting. On the contrary, a very large bandwidth over each fire occurrence doesn't capture areas where hotspots exist because the peaks have been flattened - resulting in a blurred distribution and underfitting. Grid search is implemented twice in conjunction with cross-validation - first, 12 values from 2 - 30 km were tested, then 7 values around the winner of the first grid search - the result was a 5.1 km bandwidth.

After the optimal bandwidth was found, a 1 km grid over CONUS was created and all fires within each cell were summed. The KDE was then run with the optimal bandwidth across the centroid of each grid cell and its corresponding fire occurrence count. The optimal bandwidth was chosen from each fire occurrence rather than the sum of fires per 1 km grid cell - summing fires per cell shifts each fire to the center of its cell, which is a very small shift compared to the 5 km width of the Gaussian. This results in a map that is only slightly smoother than one built from each fire occurrence, so the difference is negligible. Also, by summing fire counts per pixel we aren't losing any nuance in the fire occurrence dataset because the points within that dataset are only accurate within ~1 mi². The resulting KDE is much less expensive to rasterize than if it were smoothed over each fire occurrence point.

Iteration 2: Natural Ignitions per 1,000 km² of Fuel by Ecoregion

[12]:
```
# Natural ignitions per ecoregion
ignitions = natural_ignitions.groupby("US_L3CODE").size()
# Create a new col of ignitions per ecoregion
ecoregion["ignitions"] = ecoregion["US_L3CODE"].map(ignitions).fillna(0)

# Normalize by fuel area, not total ecoregion area: only the fuel types kept in the analysis
# (GR, GS, SH, TU, TL, Ag) count, so urban, water, barren, snow/ice and slash-blowdown land
# doesn't dilute the rate of ecoregions that contain a lot of it
fuel_km2 = (fuel_area[fuel_area["FUEL_GROUP"].isin(["GR", "GS", "SH", "TU", "TL", "Ag"])]
            .groupby("US_L3CODE")["area_km2"].sum())
ecoregion["fuel_km2"] = ecoregion["US_L3CODE"].astype(int).map(fuel_km2)
```

```
# Create a new col that contains the normalized ignition data: ignitions per 1,000 km² of fuel
ecoregion["ign_per_1000km2"] = ecoregion["ignitions"] / (ecoregion["fuel_km2"] / 1_000)

# Class breaks: the rates span 0.3-316, so each class covers roughly 2-3x the one below,
# with 4 of the 7 classes above 50 to show detail where ignitions occur. Every break falls in a
# natural gap between ecoregion rates (e.g. 150 sits in the 141 -> 188 gap).
# The last break is the maximum rate so every ecoregion falls in a class
rate_bins = [5, 15, 50, 100, 150, 200, ecoregion["ign_per_1000km2"].max()]

# Colors: one continuous yellow-to-red scale, one color per class
rate_cmap = ListedColormap(plt.get_cmap("YlOrRd")(np.linspace(0.08, 1, len(rate_bins))))

# Whole-number legend labels like "100-150" (the first class starts at 0)
rate_labels = [f"{lo:.0f}-{hi:.0f}" for lo, hi in zip([0] + rate_bins[:-1], rate_bins)]

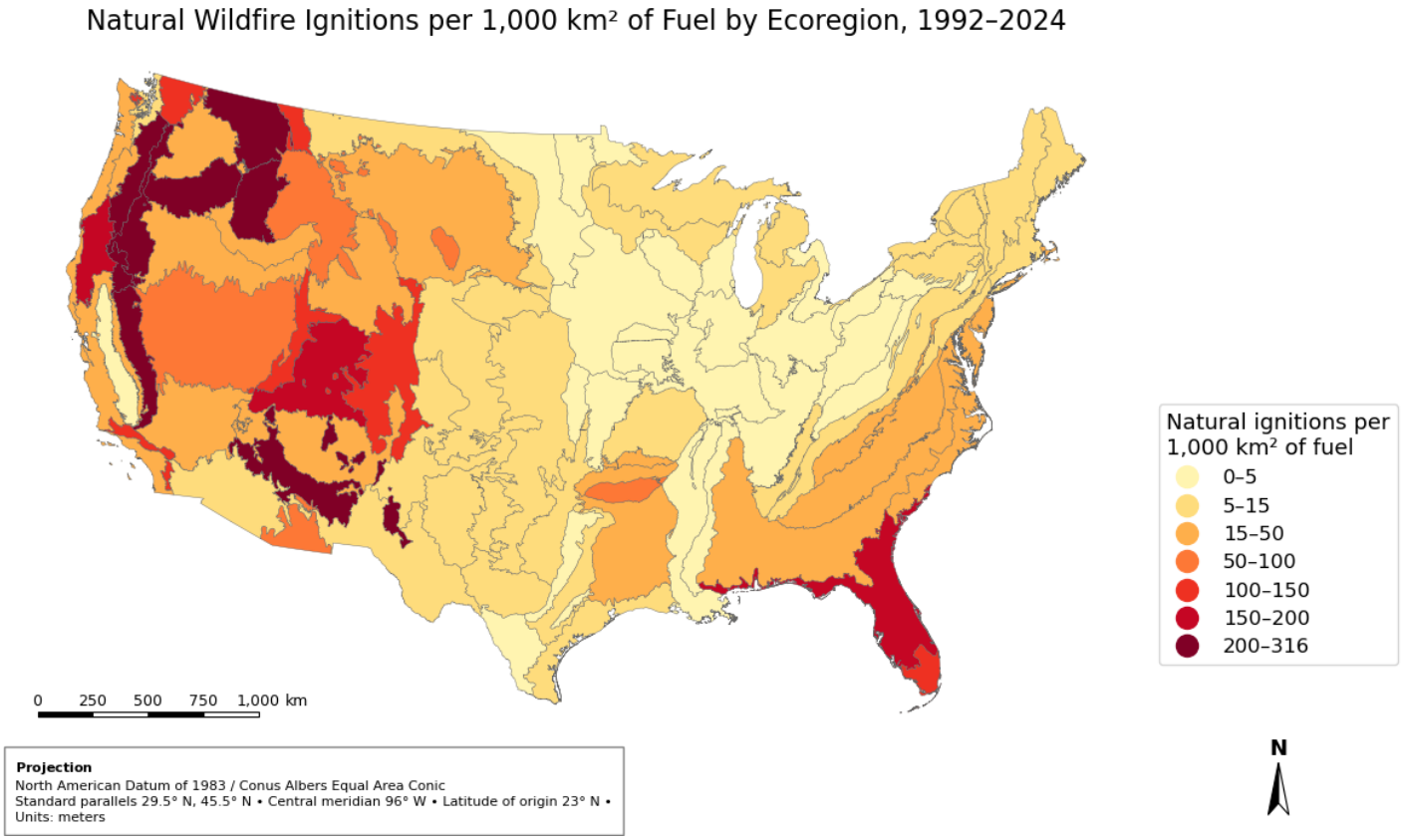
def relabel_legend(ax, labels):
    """Replace a geopandas legend's default "100.00, 150.00" labels with our own."""
    for text, label in zip(ax.get_legend().get_texts(), labels):
        text.set_text(label)

# Plot the data
fig, ax = plt.subplots(figsize=(12, 8))
ecoregion.plot(column="ign_per_1000km2", cmap=rate_cmap,
               scheme="user_defined", classification_kwds={"bins": rate_bins},
               edgecolor="0.4", linewidth=0.3, legend=True,
               # Legend placed outside the map on the right, like the colorbar on the KDE map: its
              # lower-left corner sits just past the axes' right edge, low in the frame above the north
               # arrow (bbox_to_anchor is in axes fractions), so it can't overlap the CONUS polygons.
              # "\n" wraps the title onto two lines; alignment="left" left-aligns the title with the
 ↪entries
               # fontsize / title_fontsize / markerscale enlarge the legend text and color dots
               legend_kwds={"title": "Natural ignitions per\n1,000 km² of fuel", "alignment": "left",
                            "loc": "lower left", "bbox_to_anchor": (1.01, 0.1),
                            "fontsize": 12, "title_fontsize": 13, "markerscale": 1.4},
               ax=ax)
relabel_legend(ax, rate_labels)
ax.set_title("Natural Wildfire Ignitions per 1,000 km² of Fuel by Ecoregion, 1992-2024", fontsize=16)

# Map elements laid out like the KDE map: 1,000 km scale bar (labels above) in the lower left
# with the projection box below it, north arrow below the legend in the bottom-right corner
x_lo, x_hi = ax.get_xlim()
y_lo, y_hi = ax.get_ylim()
add_scalebar(ax, x_lo + 0.02 * (x_hi - x_lo), y_lo + 0.04 * (y_hi - y_lo), length_km=1000,
             n_segments=4, height_km=25, label_every_segment=True, labels_above=True)
proj_box = add_projection_box(ax)

# North arrow: its own small axes centered under the legend, bottom level with the bottom of the
# projection box, so it sits in the figure's bottom-right corner (positions in figure fractions)
fig.canvas.draw()                       # settle the layout so the legend and box positions are known
to_fig = fig.transFigure.inverted()
leg = ax.get_legend().get_window_extent().transformed(to_fig)
box_bottom = proj_box.get_bbox_patch().get_window_extent().transformed(to_fig).y0
arrow_ax = fig.add_axes([leg.x0 + leg.width / 2 - 0.03, box_bottom, 0.06, 0.11])
arrow_ax.set_xlim(-150_000, 150_000)
arrow_ax.set_ylim(-100_000, 150_000)
arrow_ax.set_aspect("equal")
add_compass(arrow_ax, 0, 0)
arrow_ax.set_axis_off()
```

```
ax.set_axis_off()
plt.show()
```



View 3. Natural-cause wildfire ignitions per 1,000 km² of fuel for each EPA Level III ecoregion, 1992–2024. Only the fuel types kept in the analysis (grass, grass-shrub, shrub, timber-understory, timber litter and agriculture) count toward each ecoregion's fuel area. Classes are weighted toward the high end of the data.

Discussion – Second Iteration

Here we are investigating each of the 85 Level III ecoregions by dividing the number of natural ignitions in that ecoregion by the total burnable area of the region. This removes the effect of ecoregion size and unburnable land. As a result, we are able to ask "how ignition-prone is this region?", as opposed to simply "where do most of the fires start?".

There are some notable observations to be made here. The first thing to notice is that the rates are highly skewed; the median ecoregion experiences roughly 11 natural ignitions per 1,000 km², the maximum is around 316 ignitions per 1,000 km² in Arizona/New Mexico area, and over half of the ecoregions sit below 15 ignitions per 1,000 km². Only 20 regions have a fire rate over 50, and of those 20, 16 are above 100.

Another interesting observation is Florida. Even when normalizing for area, Florida still experiences a high rate of natural ignitions; interesting for a relatively wet region. This effect was not isolated to Florida, as many other iteration 1 hot spots also show up as hot spots in this iteration. As a result, we can conclude that these hot-spots aren't just showing the effect of ecoregion area, they are actually more prone to fires even after normalizing for area.

Data Processing & Visualization

- Natural ignitions are counted per ecoregion by grouping on the ecoregion code.

- Only the six kept fuel groups count toward fuel area: grass, grass-shrub, shrub, timber understory, timber litter, and agriculture.
- Rate was calculated as follows: Total ignitions / (total fuel area / 1,000 km²)
- The regions are classified into 7 classes: 5, 15, 50, 100, 150, 200, and the max at ~316; all representing the rate as noted above.
- It is colored by a single yellow-red ramp with one color per class.

Iteration 3: Spatial Weights & Moran's I

Do ecoregions with a lot of natural ignitions sit next to each other, or is the pattern on the map just noise?

The first step is to define who is next to whom. We use Queen contiguity on the 85 ecoregions, so two ecoregions are neighbors if they share any part of a boundary. The check below confirms there are no islands and that the average ecoregion has about five neighbors.

[13]:
```
# Queen contiguity: ecoregions are neighbors if they share any edge or corner
w = weights.Queen.from_dataframe(ecoregion, use_index=False)
w.transform = "R"   # row-standardize: each ecoregion's neighbor weights sum to 1

# Number of neighbors for each ecoregion
ecoregion["n_neighbors"] = pd.Series(w.cardinalities)

# How the neighbor counts are distributed across the 85 ecoregions
print(ecoregion["n_neighbors"].value_counts().sort_index())
print()

# Ecoregions with one or two neighbors.
few = ecoregion[ecoregion["n_neighbors"] <= 2]
for i, eco_row in few.iterrows():
    neighbor_names = ecoregion.loc[w.neighbors[i], "US_L3NAME"].tolist()
    print(eco_row["US_L3NAME"], "has", eco_row["n_neighbors"], "neighbor(s):", neighbor_names)
```

```
n_neighbors
1      2
2      3
3      9
4     10
5     24
6     11
7     14
8      8
9      3
10     1
Name: count, dtype: int64

Northern Minnesota Wetlands has 2 neighbor(s): ['Lake Agassiz Plain', 'Northern
Lakes and Forests']
Central California Valley has 1 neighbor(s): ['Central California Foothills and
Coastal Mountains']
Southern Florida Coastal Plain has 1 neighbor(s): ['Southern Coastal Plain']
Acadian Plains and Hills has 2 neighbor(s): ['Northeastern Highlands',
'Northeastern Coastal Zone']
Atlantic Coastal Pine Barrens has 2 neighbor(s): ['Northern Piedmont',
'Northeastern Coastal Zone']
```

The five ecoregions with only one or two neighbors are either enclosed by a single ecoregion or sit on the coast or the Canadian border, where neighbors outside the data are missing. With row standardized weights, an ecoregion with one

neighbor is compared to that single neighbor's rate, which matters later for Southern Florida.

The weights are row standardized, so each ecoregion is compared to the average of its neighbors. That average is the spatial lag, computed below.

[14]:
```
# Spatial lag of the ignition rate
# For each ecoregion, the mean rate of its neighbors.
ecoregion["lag_rate"] = weights.lag_spatial(w, ecoregion["ign_per_1000km2"].values)

# Same 7 quantile breaks on both maps so the colors are comparable
import mapclassify
bins = mapclassify.Quantiles(ecoregion["ign_per_1000km2"], k=7).bins

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

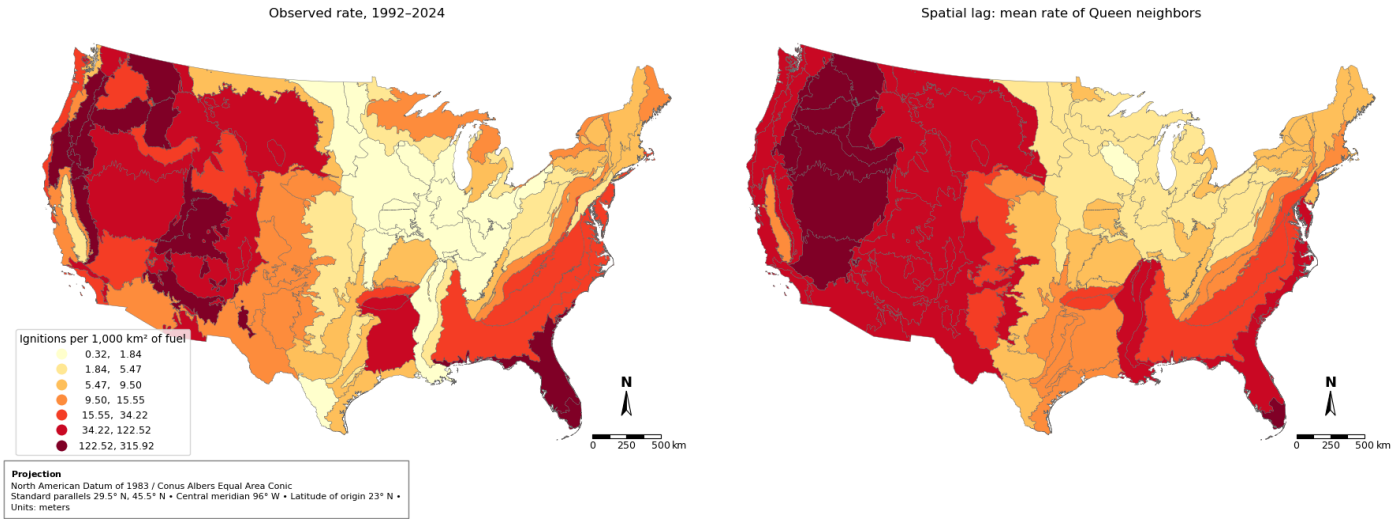
ecoregion.plot(column="ign_per_1000km2", cmap="YlOrRd",
               scheme="user_defined", classification_kwds={"bins": bins},
               edgecolor="0.4", linewidth=0.3, legend=True,
               legend_kwds={"title": "Ignitions per 1,000 km² of fuel", "loc": "lower left",
 ↪"fontsize": 9},
               ax=axes[0])
axes[0].set_title("Observed rate, 1992-2024")

ecoregion.plot(column="lag_rate", cmap="YlOrRd",
               scheme="user_defined", classification_kwds={"bins": bins},
               edgecolor="0.4", linewidth=0.3, legend=False,
               ax=axes[1])
axes[1].set_title("Spatial lag: mean rate of Queen neighbors")

# Scale bar and north arrow
for ax in axes:
    x_lo, x_hi = ax.get_xlim()
    y_lo, y_hi = ax.get_ylim()
    bar_x = x_lo + 0.84 * (x_hi - x_lo)
    bar_y = y_lo + 0.05 * (y_hi - y_lo)
    add_scalebar(ax, bar_x, bar_y, length_km=500)
    add_compass(ax, bar_x + 250_000, bar_y + 0.08 * (y_hi - y_lo))
    ax.set_axis_off()

# Projection box below the left map, like the other maps
add_projection_box(axes[0])

plt.tight_layout()
plt.show()
```



View 4. Observed ignition rate per 1,000 km² of fuel for each ecoregion (left) next to its spatial lag, the mean rate of its Queen contiguity neighbors (right). Both maps use the same 7 quantile classes, so the colors are comparable.

The right map is a smoothed version of the left one. The West turns into one dark block and the central plains into one light block, which is what positive spatial autocorrelation looks like before it is measured. Florida stands out as the one place where an ecoregion is much darker than its lag, because its only neighbor has a lower rate.

Next, global Moran's I. The rate is very skewed, from under one to over three hundred, so it is run on the log of the rate. The raw rate is run as well to check that the answer does not depend on the transform.

[15]:
```
# Global Moran's I on the ignition rate
from esda.moran import Moran

rate = ecoregion["ign_per_1000km2"].values
print("Rate range:", round(rate.min(), 2), "to", round(rate.max(), 2))

# Use the log rate.
log_rate = np.log(rate)

np.random.seed(12345)
mi_log = Moran(log_rate, w, permutations=999)
mi_raw = Moran(rate, w, permutations=999)

print("Moran's I (log rate):", round(mi_log.I, 3), " p:", mi_log.p_sim)
print("Moran's I (raw rate):", round(mi_raw.I, 3), " p:", mi_raw.p_sim)
print("Expected I if no spatial autocorrelation:", round(mi_log.EI, 3))
```

```
Rate range: 0.32 to 315.92
Moran's I (log rate): 0.616  p: 0.001
Moran's I (raw rate): 0.387  p: 0.001
Expected I if no spatial autocorrelation: -0.012
```

The next cell checks the same result another way. Each ecoregion is placed in one of the four Moran quadrants by comparing its own standardized rate with the average of its neighbors, and the slope of the spatial lag on the standardized rate should reproduce Moran's I.

[16]:
```
# z: standardized log rate of each ecoregion
# lag_z: the mean standardized log rate of its neighbors
z = (log_rate - log_rate.mean()) / log_rate.std()
lag_z = weights.lag_spatial(w, z)
```

```
ecoregion["z_log_rate"] = z
ecoregion["lag_z_log_rate"] = lag_z

# Check the slope of lag_z
slope = np.sum(z * lag_z) / np.sum(z * z)
print("Slope of lag_z on z:", round(slope, 3), " Moran's I:", round(mi_log.I, 3))

# Which quadrant of the Moran scatterplot each ecoregion falls in
ecoregion["moran_quadrant"] = "High-Low"
ecoregion.loc[(z > 0) & (lag_z > 0), "moran_quadrant"] = "High-High"
ecoregion.loc[(z < 0) & (lag_z > 0), "moran_quadrant"] = "Low-High"
ecoregion.loc[(z < 0) & (lag_z < 0), "moran_quadrant"] = "Low-Low"
print(ecoregion["moran_quadrant"].value_counts())
print()

# Ecoregions that sit far from the rest
far = ecoregion[(np.abs(z) > 2) | (np.abs(lag_z) > 1.5)]
display(far[["US_L3NAME", "ign_per_1000km2", "z_log_rate", "lag_z_log_rate", "moran_quadrant"]])
```

```
Slope of lag_z on z: 0.616  Moran's I: 0.616
moran_quadrant
Low-Low      39
High-High    30
High-Low      9
Low-High      7
Name: count, dtype: int64


                        US_L3NAME  ign_per_1000km2  z_log_rate  \
1                 Columbia Plateau        23.308222    0.312966
47                  Driftless Area         1.835001   -1.158522
49        Central Corn Belt Plains         0.317073   -2.174921
73  Southern Florida Coastal Plain       140.623843    1.353457

    lag_z_log_rate moran_quadrant
1         1.592452      High-High
47       -1.551792        Low-Low
49       -1.166260        Low-Low
73        1.544723      High-High
```

Global Moran's I only says clustering exists. Local Moran's I says where. Each ecoregion is tested against its own neighbors, and the ones that are significant at 0.05 keep their quadrant label.

[17]:
```
# Local Moran's I tests each ecoregion against its own neighbors.
from esda.moran import Moran_Local

lisa = Moran_Local(log_rate, w, permutations=999, seed=12345)

# lisa.q gives the Moran quadrant: 1 High-High, 2 Low-High, 3 Low-Low, 4 High-Low
# lisa.p_sim is the permutation pseudo p value for each ecoregion
ecoregion["lisa_I"] = lisa.Is
ecoregion["lisa_p"] = lisa.p_sim
ecoregion["lisa_q"] = lisa.q

# An ecoregion significant at alpha keeps its quadrant label, the rest are Not significant
alpha = 0.05
labels = {1: "High-High", 2: "Low-High", 3: "Low-Low", 4: "High-Low"}
ecoregion["lisa_cluster"] = "Not significant"
sig = ecoregion["lisa_p"] < alpha
ecoregion.loc[sig, "lisa_cluster"] = ecoregion.loc[sig, "lisa_q"].map(labels)
```

```
# How many ecoregions fall in each cluster type
print(ecoregion["lisa_cluster"].value_counts())
print()

# Table of the significant ecoregions, grouped by cluster type and sorted by local I
lisa_table = ecoregion.loc[sig, ["US_L3CODE", "US_L3NAME", "ign_per_1000km2", "lag_rate",
                                 "lisa_I", "lisa_p", "lisa_cluster"]]
lisa_table = lisa_table.sort_values(["lisa_cluster", "lisa_I"], ascending=[True, False])
display(lisa_table)
```

```
lisa_cluster
Not significant    50
High-High          19
Low-Low            15
Low-High            1
Name: count, dtype: int64
```

	US_L3CODE	US_L3NAME	\
6	15	Northern Rockies	
84	9	Eastern Cascades Slopes and Foothills	
2	11	Blue Mountains	
7	16	Idaho Batholith	
12	20	Colorado Plateaus	
44	5	Sierra Nevada	
33	4	Cascades	
4	13	Central Basin and Range	
10	19	Wasatch and Uinta Mountains	
75	78	Klamath Mountains/California High North Coast …	
8	17	Middle Rockies	
74	77	North Cascades	
78	80	Northern Basin and Range	
3	12	Snake River Plain	
9	18	Wyoming Basin	
5	14	Mojave Basin and Range	
14	22	Arizona/New Mexico Plateau	
1	10	Columbia Plateau	
55	6	Central California Foothills and Coastal Mount…	
79	81	Sonoran Basin and Range	
49	54	Central Corn Belt Plains	
69	72	Interior River Valleys and Hills	
41	47	Western Corn Belt Plains	
47	52	Driftless Area	
48	53	Southeastern Wisconsin Till Plains	
40	46	Northern Glaciated Plains	
50	55	Eastern Corn Belt Plains	
67	70	Western Allegheny Plateau	
68	71	Interior Plateau	
34	40	Central Irregular Plains	
20	28	Flint Hills	
46	51	North Central Hardwood Forests	
32	39	Ozark Highlands	
51	56	Southern Michigan/Northern Indiana Drift Plains	
45	50	Northern Lakes and Forests	

	ign_per_1000km2	lag_rate	lisa_I	lisa_p	lisa_cluster
6	247.291643	143.688943	1.976778	0.003	High-High
84	237.435224	128.887781	1.868897	0.002	High-High
2	246.142153	140.612414	1.811799	0.002	High-High

7	256.949416	126.834545	1.756264	0.005	High-High
12	199.913660	105.832428	1.535598	0.004	High-High
44	241.937844	88.909208	1.427385	0.015	High-High
33	215.358177	111.083124	1.364878	0.004	High-High
4	64.653419	157.987027	1.152570	0.001	High-High
10	130.453669	77.685729	1.146065	0.007	High-High
75	188.314810	100.397497	1.076172	0.041	High-High
8	86.512055	99.379146	0.909824	0.002	High-High
74	116.666357	100.825855	0.870214	0.043	High-High
78	47.771325	132.685862	0.851409	0.004	High-High
3	30.918653	159.343737	0.609935	0.004	High-High
9	34.224621	114.862586	0.609462	0.005	High-High
5	33.039294	140.321668	0.580448	0.002	High-High
14	40.012249	115.806408	0.513874	0.017	High-High
1	23.308222	220.896959	0.492520	0.001	High-High
55	15.292548	114.501927	0.053143	0.014	High-High
79	13.475373	94.204790	-0.003178	0.026	Low-High
49	0.317073	2.794563	2.506683	0.001	Low-Low
69	0.631794	1.976934	2.386086	0.001	Low-Low
41	0.458444	3.904647	1.806047	0.001	Low-Low
47	1.835001	1.595118	1.776635	0.001	Low-Low
48	1.495598	2.408251	1.641858	0.008	Low-Low
40	1.367482	3.112286	1.580347	0.009	Low-Low
50	1.286526	3.943861	1.450821	0.001	Low-Low
67	1.912441	2.784124	1.132962	0.002	Low-Low
68	1.459009	6.299886	1.089196	0.018	Low-Low
34	1.132278	7.272196	1.059809	0.016	Low-Low
20	2.429128	3.957803	1.032851	0.018	Low-Low
46	5.072680	3.622692	0.639674	0.001	Low-Low
32	6.258806	6.877067	0.422665	0.013	Low-Low
51	8.723895	4.501596	0.244007	0.015	Low-Low
45	10.916198	4.703464	0.098024	0.040	Low-Low

Nineteen ecoregions come out High-High, and they form one block across the interior West, from the Cascades and the Northern Rockies down to the Colorado Plateau and the Mojave. Fifteen come out Low-Low across the Corn Belt and the central plains. The Sonoran desert is the one Low-High, a low spot inside the western cluster. No ecoregion is High-Low.

Discussion – Third Iteration

The spatial statistics confirm what the choropleth suggests by eye. Natural ignition rates are not scattered at random across ecoregions. Global Moran's I on the log of the rate is 0.616 with a permutation p value of 0.001, and the local results put 19 ecoregions in one High-High block across the interior West and 15 in a Low-Low block across the Corn Belt and the central plains. For the research question this means the distribution of natural fires is organized at a scale larger than a single ecoregion. Neighboring ecoregions tend to share the same fire regime, which points to drivers that cross ecoregion boundaries, such as summer lightning and terrain, rather than something unique to each region.

A few limits matter for reading these results. The analysis works at the ecoregion level, so it says nothing about variation inside an ecoregion, and ecoregions differ greatly in size, which is a modifiable areal unit problem. The weights stop at the national border, so ecoregions on the coast or along Canada have fewer neighbors than they really do. Southern Florida shows this clearly. It has one of the highest rates in the East but only one neighbor, so the local test has almost no power there and it is not flagged. With 85 local tests at a cutoff of 0.05 a few of the significant ecoregions are expected to be false positives, and a couple of the High-High members, such as the Columbia Plateau, are in the cluster because of their neighbors rather than their own rate.

Iteration 4: Natural Ignition Density by Fuel Group

[18]:
```
# Fuel group = first two letters of the FBFM40 code (GR, GS, SH, TU, TL); NB3 agriculture -> "Ag"
# Create a dictionary of fuel class and the corresponding label
code_to_label = dict(zip(fuel_lookup["VALUE"], fuel_lookup["FBFM40"]))
# Replace each fires numeric code with its fuel-type label
labels = natural_ignitions["FUEL_CLASS"].map(code_to_label)
# Group each fuel class by fuel group (drop the third char from the Scott and Burgan labels (GR1, GR2
 ↪to GR))
natural_ignitions["FUEL_GROUP"] = labels.str[:2].replace({"NB": None})
# Any NB3 label becomes Ag
natural_ignitions.loc[labels == "NB3", "FUEL_GROUP"] = "Ag"

# Create an empty dictionary to store density grid by fuel-class
density_by_group = {}
# groupby splits the table by FUEL_GROUP. Each pass through the loop gives you group and fires
for group, fires in natural_ignitions.groupby("FUEL_GROUP"):
    # Blank 1km grid is created (nrows and ncols come from step 1 when creating grid)
    counts_g = np.zeros((nrows, ncols), dtype="float32")
    # Adds 1 to the cell of every fire in this group
    np.add.at(counts_g, (fires["ROW"].to_numpy(), fires["COL"].to_numpy()), 1)
    # Smooths the counts into a density surface
    dens = gaussian_filter(counts_g, sigma=sigma_cells, mode="constant", cval=0.0)
    # Blanks out cells outside CONUS so they don't get counted
    dens[~conus_mask] = np.nan
    # Saves the group's surface in the dictionary under its name
    density_by_group[group] = dens

    print(f"{group}: {len(fires):,} fires")
```

```
Ag: 4,160 fires
GR: 64,053 fires
GS: 69,233 fires
SH: 44,758 fires
TL: 70,376 fires
TU: 66,568 fires
```

[19]:
```
groups = ["GR", "GS", "SH", "TU", "TL", "Ag"]
names = {"GR": "Grass", "GS": "Grass-shrub", "SH": "Shrub", "TU": "Timber-understory",
         "TL": "Timber litter", "Ag": "Agriculture"}
extent = (xmin, xmin + ncols * cell_size, ymax - nrows * cell_size, ymax)
norm = LogNorm(vmin=1e-3, vmax=max(np.nanmax(d) for d in density_by_group.values()))

# Densities below 0.001 per km² (including zero) are drawn transparent so the grey CONUS fill
# underneath shows through: grey = (almost) no ignitions, pale yellow = few ignitions
panel_cmap = plt.get_cmap("YlOrRd").copy()
panel_cmap.set_under("none")

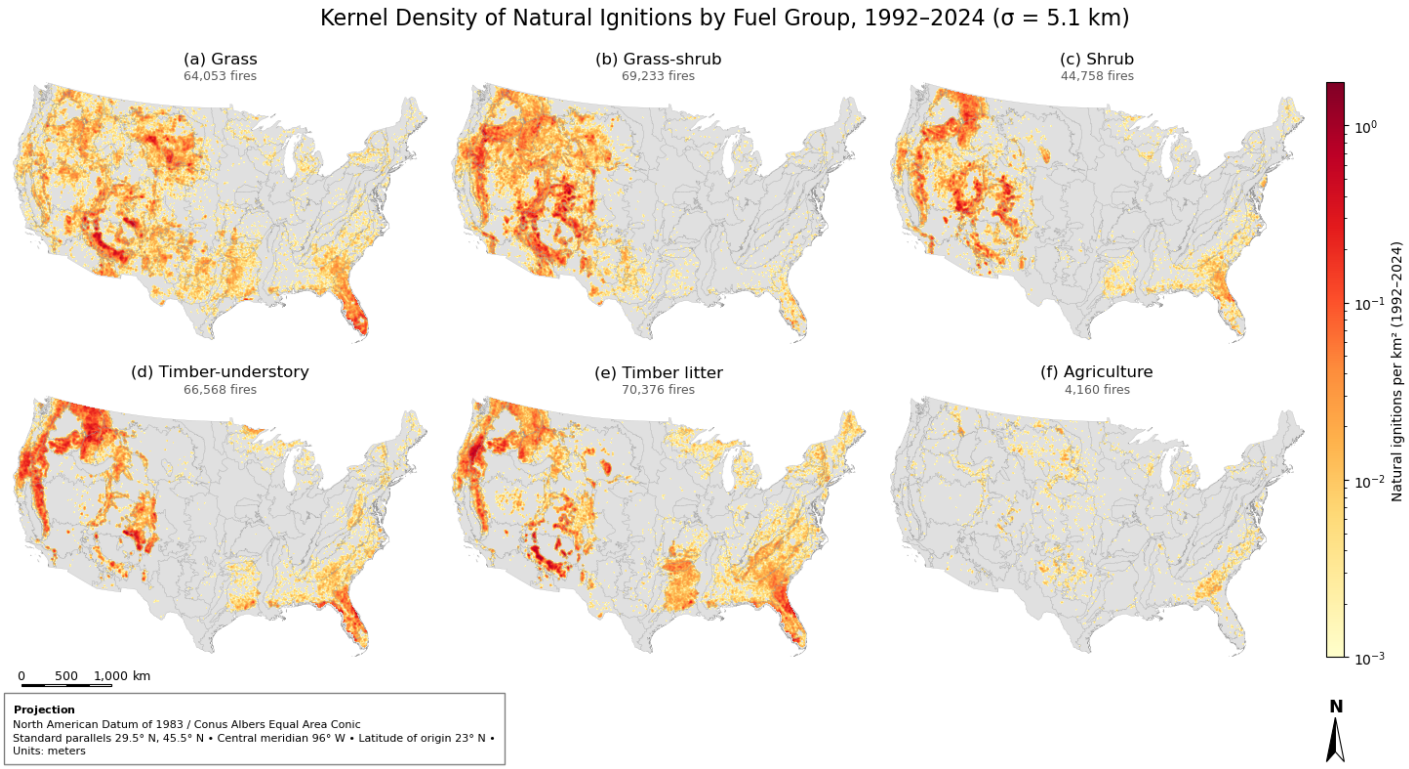
# 6 groups -> 2 rows x 3 columns. Figure size and small gaps are chosen so the wide CONUS maps
# fill their panels; right=0.9 leaves room for the colorbar
fig, axes = plt.subplots(2, 3, figsize=(15, 7))
fig.subplots_adjust(left=0.01, right=0.9, top=0.87, bottom=0.03, wspace=0.03, hspace=0.2)
for ax, g, letter in zip(axes.flat, groups, "abcdef"):
    # zorder stacks the layers: grey CONUS fill, then density, then light, thin ecoregion lines
    ecoregion.plot(ax=ax, color="0.88", linewidth=0, zorder=1)
    im = ax.imshow(density_by_group[g], extent=extent, cmap=panel_cmap, norm=norm, zorder=2)
    ecoregion.boundary.plot(ax=ax, color="0.6", linewidth=0.12, zorder=3)
    # Panel letter and group name, with the fire count on a smaller second line just below it
    ax.set_title(f"({letter}) {names[g]}", fontsize=12, pad=14)
```

```
    ax.text(0.5, 1.0, f"{(natural_ignitions['FUEL_GROUP'] == g).sum():,} fires", transform=ax.
 ↪transAxes,
            ha="center", va="bottom", fontsize=9, color="0.35")
    ax.set_axis_off()
fig.suptitle(f"Kernel Density of Natural Ignitions by Fuel Group, 1992-2024 (σ = {bandwidth / 1000:.
 ↪1f} km)",
             fontsize=16)

# One set of map elements for the whole figure: scale bar below the maps in the bottom-left corner
# with the projection box under it, colorbar the full height of both rows, north arrow below the
 ↪colorbar
fig.canvas.draw()                       # settle the layout so the panel positions are known
to_fig = fig.transFigure.inverted()

# The scale bar gets its own strip of axes just below panel (d), with the same width and x/y limits
# per figure fraction as the panel, so 1,000 km on the bar is 1,000 km on the maps
panel = axes[1, 0].get_position()
x_lo, x_hi = axes[1, 0].get_xlim()
y_lo, y_hi = axes[1, 0].get_ylim()
strip_h = 0.05                                                  # strip height in figure fractions
sb_ax = fig.add_axes([panel.x0, panel.y0 - strip_h, panel.width, strip_h])
sb_ax.set_xlim(x_lo, x_hi)
sb_ax.set_ylim(0, strip_h * (y_hi - y_lo) / panel.height)       # same meters per figure fraction as
 ↪the map
sb_ax.set_axis_off()
# Labels only at 0 / 500 / 1,000 km: every segment boundary is too crowded at panel size
add_scalebar(sb_ax, x_lo + 0.02 * (x_hi - x_lo), 0.15 * strip_h * (y_hi - y_lo) / panel.height,
             length_km=1000, n_segments=4, height_km=25, labels_above=True)
proj_box = add_projection_box(sb_ax, y=-0.35)                   # hangs just below the scale bar strip
fig.canvas.draw()                       # settle again so the projection box position is known
top_panel, bottom_panel = axes[0, 2].get_position(), axes[1, 2].get_position()
cax = fig.add_axes([top_panel.x1 + 0.015, bottom_panel.y0, 0.012, top_panel.y1 - bottom_panel.y0])
cbar = fig.colorbar(im, cax=cax, label="Natural ignitions per km² (1992-2024)")

# North arrow: its own small axes centered under the colorbar, bottom level with the bottom of the
# projection box, so it sits in the figure's bottom-right corner (positions in figure fractions)
cb = cax.get_position()
box_bottom = proj_box.get_bbox_patch().get_window_extent().transformed(to_fig).y0
# min() drops the arrow further if needed so its "N" clears the colorbar's bottom tick label
arrow_ax = fig.add_axes([cb.x0 + cb.width / 2 - 0.03, min(box_bottom, cb.y0 - 0.13), 0.06, 0.09])
arrow_ax.set_xlim(-150_000, 150_000)
arrow_ax.set_ylim(-100_000, 150_000)
arrow_ax.set_aspect("equal")
add_compass(arrow_ax, 0, 0)
arrow_ax.set_axis_off()
plt.show()
```



View 5. Kernel density of natural-cause wildfire ignitions for each fuel group, using the same cross-validated bandwidth as View 2. All panels share one log color scale, so the same color means the same density in every panel. Grey areas have fewer than 0.001 natural ignitions per km². Grey lines are EPA Level III ecoregion boundaries.

Discussion – Fourth Iteration

Ranking the fuel types by total fire count, in descending order, goes as follows:

- Timber-litter: 70,376
- Grass-Shrub: 69,233
- Timber-understory: 66,568
- Grass: 64,053
- Shrub: 44,758
- Agriculture: 4,160

By order of the visuals:

- (a) Grass. Notable hotspots for grass show up in Arizona/New Mexico and Florida.
- (b) Grass-shrub. Notable hotspots for grass-shrub are concentrated in the West, with a few outlying points in Florida.
- (c) Shrub. Very similar to grass-shrub.
- (d) Timber-understory. There is a high concentration of timber-understory fires in the Northwest, Rocky Mountain region, and the forested areas of California.
- (e) Timber-litter. Very similar pattern to timber-understory, though this fuel type also shows a high concentration in the East, unlike many of the other fuel types.
- (f) Agriculture. Agriculture areas show very little fire activity. There are faint patches in the Great Plains and Mississippi Delta area. Although it covers roughly 15% of the total fuel area, it only accounts for around 1.3% of the total fires.

Across all fuel types, we see that the Central Midwest and Corn Belt areas are the least prone to fires. They are grey/uncolored in nearly every panel.

Data Processing & Visualization

Fuel Grouping

- Extracted fuel groups (GR, GS, SH, TU, TL) by their two letter prefix; NB3 (agriculture) became its own group, Ag

Iteration 5: Dominant Fuel Group by Ignition Density

[20]:
```
# Track the highest-density group in each cell, one group at a time (saves memory vs. stacking)
best = np.zeros((nrows, ncols), dtype="float32")
dominant = np.full((nrows, ncols), -1, dtype="int8")
for i, g in enumerate(groups):
    d = np.nan_to_num(density_by_group[g])
    higher = d > best
    best[higher], dominant[higher] = d[higher], i

dominant = np.where(best >= 1e-3, dominant, -1)   # blank cells with almost no ignitions
dominant_masked = np.ma.masked_equal(dominant, -1)

colors = ["#e6c84f", "#9cc97a", "#c8743c", "#4fa3a0", "#2f5d3a", "#f2a7b8"]

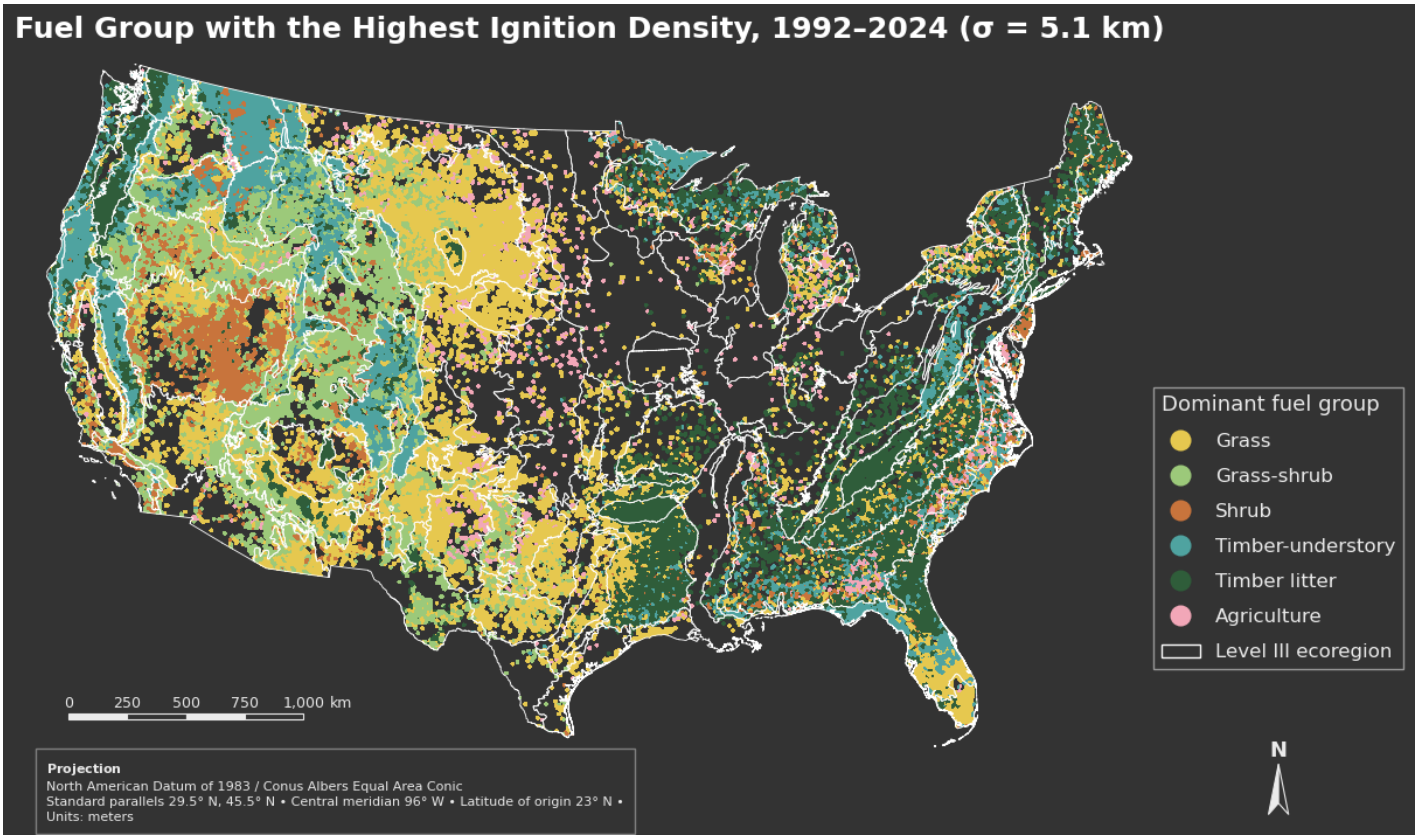
# Styled like the overview map: dark grey background, bold white title, light map elements
bg = "#333333"   # dark grey background
fg = "0.92"      # near-white for text, outlines and map elements
fig, ax = plt.subplots(figsize=(12, 8), facecolor=bg)   # axis is turned off, so the figure color is
 ↪what shows
ax.imshow(dominant_masked, extent=extent, cmap=ListedColormap(colors),
          vmin=-0.5, vmax=len(groups) - 0.5, interpolation="nearest")
# White outlines: no fuel group color is near-white, so they stand out from both the grey and the cells
ecoregion.boundary.plot(ax=ax, color="white", linewidth=0.6)
ax.set_title(f"Fuel Group with the Highest Ignition Density, 1992-2024 (σ = {bandwidth / 1000:.1f}
 ↪km)",
             color="white", fontsize=18, fontweight="bold", pad=16)

# Legend low on the right, just outside the map so it doesn't cover the Southeast: circle markers
# for each fuel group plus an outlined box for the ecoregion boundaries, like the overview map
handles = [Line2D([], [], marker="o", linestyle="none", markersize=13, markerfacecolor=c,
                  markeredgecolor="none", label=names[g]) for c, g in zip(colors, groups)]
handles.append(Patch(facecolor="none", edgecolor="white", label="Level III ecoregion"))
leg = ax.legend(handles=handles, loc="lower left", bbox_to_anchor=(1.01, 0.1), title="Dominant fuel
 ↪group",
                fontsize=12, title_fontsize=13, alignment="left", labelspacing=0.9,
                facecolor=bg, edgecolor="0.6", labelcolor=fg, framealpha=1, fancybox=False)
leg.get_title().set_color(fg)

# Map elements: 1,000 km scale bar (labels above) in the lower left with the projection box below it
x_lo, x_hi = ax.get_xlim()
y_lo, y_hi = ax.get_ylim()
add_scalebar(ax, x_lo + 0.02 * (x_hi - x_lo), y_lo + 0.04 * (y_hi - y_lo), length_km=1000,
             n_segments=4, height_km=25, label_every_segment=True, labels_above=True,
             fill=(fg, bg), color=fg)
proj_box = add_projection_box(ax, color=fg, facecolor=bg)

# North arrow: its own small axes centered under the legend, bottom level with the bottom of the
# projection box, so it sits in the figure's bottom-right corner (positions in figure fractions)
fig.canvas.draw()                       # settle the layout so the legend and box positions are known
```

```
to_fig = fig.transFigure.inverted()
leg_box = leg.get_window_extent().transformed(to_fig)
box_bottom = proj_box.get_bbox_patch().get_window_extent().transformed(to_fig).y0
arrow_ax = fig.add_axes([leg_box.x0 + leg_box.width / 2 - 0.03, box_bottom, 0.06, 0.11])
arrow_ax.set_xlim(-150_000, 150_000)
arrow_ax.set_ylim(-100_000, 150_000)
arrow_ax.set_aspect("equal")
add_compass(arrow_ax, 0, 0, fg=fg, bg=bg)
arrow_ax.set_axis_off()
ax.set_axis_off()
plt.show()
```



View 6. Fuel group with the highest natural ignition density in each 1 km cell, based on the fuel-group surfaces in View 5. Cells with almost no ignitions are left blank.

Discussion – Fifth Iteration

- Visualizing each 1km cell by its most prominent fuel type, we are able to see which fuel types contribute to the most number of fires by area.
- The widest footprint of these fuel types seems to be grass, covering a large area in the Great Plains down through Texas and Oklahoma, along with a good chunk of the Florida Peninsula.
- Timber litter seems to be the most prominent fuel type in the east, comprising a large area there and very little elsewhere.
- Grass shrub seems to cluster around the Rocky Mountain region.
- Shrub seems to be confined to a small cluster in the Nevada/Utah area, with very few points scattered elsewhere.
- Timber understory, similar to grass shrub, seems to cluster around the mountainous regions of the West, along with a few patches in the mountainous regions of the East.

- Agriculture is scattered lightly around the country, with only a slightly higher concentration in the East.

Data Processing & Visualization

- For each 1 km² cell, the fuel type with the highest number of ignitions within a 5.1km radius is selected.
- Cells whose highest density is below 0.001/km² are set to none to avoid insignificant noise.
- It is colored categorically by dominant fuel type, as denoted by the legend.

Note: because the input is a ~5km radius, each cell's color represents the dominant fuel type around the cell, not necessarily the region confined to the cell itself.

Iteration 6: Ignition Rate by Fuel Group

Which fuels are most receptive to ignition?

Ignition rate = natural ignitions on a fuel group ÷ area of that fuel group ÷ number of years, expressed as ignitions per 1,000 km² of fuel per year. This normalizes ignitions by how much of each fuel exists across CONUS, so a fuel that covers a greater area isn't favored.

[21]:
```
# Now that we have the area of each fuel type (per ecoregion, summed here across CONUS),
# we can normalize: ignitions on each fuel type ÷ area of that fuel type ÷ years
# = ignitions per 1,000 km² of fuel per year, so fuels that cover more ground aren't favored

n_years = 2024 - 1992 + 1

# Ignitions per fuel group (natural_ignitions exclude urban, snow/ice, water, barren, slash-
blowdown, no-data)
ignitions_by_group = natural_ignitions.groupby("FUEL_GROUP").size()

# Fuel area per fuel group, summed over all ecoregions (km²)
area_by_group = fuel_area.groupby("FUEL_GROUP")["area_km2"].sum()

# Create a table, one row per group; only groups in `groups` (same order/colors as the maps)
receptivity = pd.DataFrame({"ignitions": ignitions_by_group, "area_km2": area_by_group}).loc[groups]

# Rate: ignitions / area gives per km² over all years
receptivity["rate"] = receptivity["ignitions"] / receptivity["area_km2"] * 1_000 / n_years

# 95% confidence interval for the rate: we are 95% confident that the true ignition rate on each
# fuel type falls within this interval (natural ignitions per 1,000 km² of that fuel per year).
# It reflects random variation in the fire counts.
n = receptivity["ignitions"]
scale = 1_000 / (receptivity["area_km2"] * n_years)
# lower limit
receptivity["rate_low"] = chi2.ppf(0.025, 2 * n) / 2 * scale
# upper limit
receptivity["rate_high"] = chi2.ppf(0.975, 2 * n + 2) / 2 * scale

plot_df = receptivity.sort_values("rate")   # sorted so the most receptive bar is on top

print(receptivity.round(3))

# Horizontal bar chart: one bar per fuel group, colored like the maps, with 95% CI error bars
group_color = dict(zip(groups, colors))
fig, ax = plt.subplots(figsize=(9, 5))
ax.barh([names[g] for g in plot_df.index],
        plot_df["rate"],
        xerr=[plot_df["rate"] - plot_df["rate_low"],
```

```
            plot_df["rate_high"] - plot_df["rate"]],
        color=[group_color[g] for g in plot_df.index],
        edgecolor="0.3", capsize=3)
ax.set_xlabel("Natural ignitions per 1,000 km² of fuel per year")
ax.set_title("Ignition Rate by Fuel Group, CONUS 1992–2024")
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.show()
```

FUEL_GROUP	ignitions	area_km2	rate	rate_low	rate_high
GR	64053.0	2224220.838	0.873	0.866	0.879
GS	69233.0	1043392.110	2.011	1.996	2.026
SH	44758.0	525452.384	2.581	2.557	2.605
TU	66568.0	673193.543	2.996	2.974	3.019
TL	70376.0	1431520.427	1.490	1.479	1.501
Ag	4160.0	1168188.522	0.108	0.105	0.111

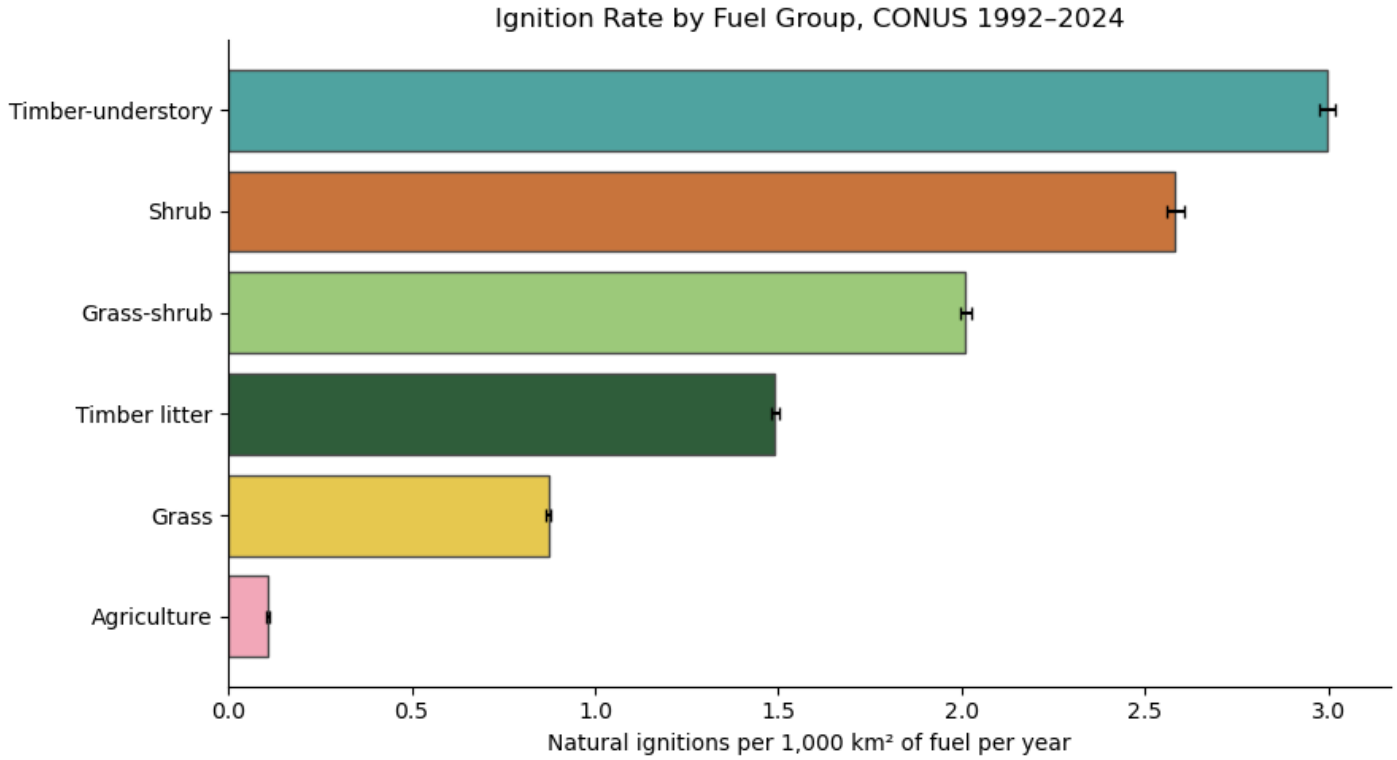


Figure 1. Natural ignitions per 1,000 km² of fuel per year for each fuel group across CONUS, 1992–2024. Error bars are Poisson 95% confidence intervals.

Discussion – Final Iteration

With this final iteration, we can finally answer the question “What fuel types are most receptive to natural ignition?”.

Ranking the fuel types by the highest rate of natural ignitions per 1,000 km²/year, we see that timber-understory is the most receptive to natural ignitions, at a rate of roughly 3.0. The rate decreases by about 0.5 with each consecutive fuel group, though agriculture sits disproportionately lower than the rest and does not follow this trend.

Data Processing & Visualization

- Ignitions per fuel group come from the cleaned ignition table. Fuel area per group comes from the 30 m fuel-raster tally summed across all ecoregions.
- Rate is calculated as follows: Ignitions ÷ area × 1,000 ÷ 33 years
- Ignition counts are treated as Poisson counts. The exact interval comes from chi-square quantiles, then is scaled to the same per-1,000 km² per-year units.
- It is then plotted as a horizontal bar chart sorted by rate so that the most receptive fuels is on the top, with error bars for each rate.

Summary tables for the interactive visuals

The interactive charts only need counts, not every fire, so everything below is built from three small tables: - `fire_summary`: natural ignitions counted by ecoregion × fuel group × year × month × fire size class. Yearly trends, seasonality, and size breakdowns are all a `groupby` of this table. - `eco_fuel`: one row per ecoregion × fuel group with ignitions, fuel area, ignition rate (95% CI), and each fuel's share of the ecoregion's ignitions and fuel area - `eco_summary`: one row per ecoregion with its totals, overall ignition rate, and the fuel group with the most ignitions

Relative rate = share of the ecoregion's ignitions ÷ share of its fuel area. Above 1 means that fuel ignites more than its area alone would predict *within that ecoregion*; below 1 means less.

[22]:
```
# NWCG fire size classes (acres): A < 0.25, B 0.25-9.9, C 10-99.9, D 100-299, E 300-999, F 1,000-
4,999, G 5,000+
size_bins = [0, 0.25, 10, 100, 300, 1_000, 5_000, np.inf]

# One row per fire with only the attributes the summaries need
fire_attrs = pd.DataFrame({
    "US_L3CODE": natural_ignitions["US_L3CODE"].astype(int),
    "FUEL_GROUP": natural_ignitions["FUEL_GROUP"],
    "FIRE_YEAR": natural_ignitions["FIRE_YEAR"],
    "MONTH": pd.to_datetime(natural_ignitions["DISCOVERY_DATE"]).dt.month.astype("int8"),
    "SIZE_CLASS": pd.cut(natural_ignitions["FIRE_SIZE"], size_bins, labels=list("ABCDEFG"),
 ↪right=False),
    "FIRE_SIZE": natural_ignitions["FIRE_SIZE"],
})

# Table 1: counts and acres burned per ecoregion / fuel group / year / month / size class
# observed=True keeps only combinations that actually have fires (SIZE_CLASS is categorical)
fire_summary = (fire_attrs
                .groupby(["US_L3CODE", "FUEL_GROUP", "FIRE_YEAR", "MONTH", "SIZE_CLASS"],
 ↪observed=True)
                .agg(ignitions=("FIRE_SIZE", "size"), acres_burned=("FIRE_SIZE", "sum"))
                .reset_index())

# Ecoregion names and Level I region (title case reads better than the shapefile's ALL CAPS)
eco_info = pd.DataFrame({"US_L3CODE": ecoregion["US_L3CODE"].astype(int),
                         "US_L3NAME": ecoregion["US_L3NAME"],
                         "NA_L1NAME": ecoregion["NA_L1NAME"].str.title()}).set_index("US_L3CODE").
 ↪sort_index()

# Table 2: every ecoregion × fuel group pair, including pairs with no fires or no fuel (filled with 0)
pairs = pd.MultiIndex.from_product([eco_info.index, groups], names=["US_L3CODE", "FUEL_GROUP"])
eco_fuel = pd.DataFrame({
    "ignitions": fire_summary.groupby(["US_L3CODE", "FUEL_GROUP"])["ignitions"].sum(),
    "area_km2": fuel_area.groupby(["US_L3CODE", "FUEL_GROUP"])["area_km2"].sum(),
}).reindex(pairs).fillna(0).astype({"ignitions": "int64"})
```

```
# Ignition rate and 95% CI, same method as the CONUS-wide bar chart above.
# Area of 0 becomes NaN so pairs where the fuel doesn't exist get a NaN rate instead of dividing by
 ↪zero
n = eco_fuel["ignitions"]
scale = 1_000 / (eco_fuel["area_km2"].where(eco_fuel["area_km2"] > 0) * n_years)
eco_fuel["rate"] = n * scale
eco_fuel["rate_low"] = np.where(n > 0, chi2.ppf(0.025, 2 * n) / 2, 0) * scale   # lower limit is 0
 ↪when there are no fires
eco_fuel["rate_high"] = chi2.ppf(0.975, 2 * n + 2) / 2 * scale

# Each fuel's share of its ecoregion's ignitions and fuel area, and their ratio (relative rate)
eco_totals = eco_fuel.groupby(level="US_L3CODE")[["ignitions", "area_km2"]].transform("sum")
eco_fuel["ign_share"] = eco_fuel["ignitions"] / eco_totals["ignitions"]
eco_fuel["area_share"] = eco_fuel["area_km2"] / eco_totals["area_km2"]
eco_fuel["rel_rate"] = eco_fuel["ign_share"] / eco_fuel["area_share"]

# Table 3: one row per ecoregion
eco_summary = eco_info.assign(
    ignitions=eco_fuel.groupby(level="US_L3CODE")["ignitions"].sum(),
    burnable_km2=eco_fuel.groupby(level="US_L3CODE")["area_km2"].sum(),
)
eco_summary["rate"] = eco_summary["ignitions"] / eco_summary["burnable_km2"] * 1_000 / n_years
eco_summary["top_fuel_group"] = eco_fuel["ignitions"].unstack()[groups].idxmax(axis=1)

# Sanity check: every fire should be counted exactly once
print(f"Fires in fire_summary: {fire_summary['ignitions'].sum():,} (should equal
 ↪{len(natural_ignitions):,})")
print(f"Fires in eco_fuel:     {eco_fuel['ignitions'].sum():,}")
display(fire_summary.head())
display(eco_fuel.head(6).round(3))
display(eco_summary.sort_values("rate", ascending=False).head().round(2))
```

```
Fires in fire_summary: 319,148 (should equal 319,148)
Fires in eco_fuel:     319,148
```

	US_L3CODE	FUEL_GROUP	FIRE_YEAR	MONTH	SIZE_CLASS	ignitions	acres_burned
0	1	Ag	1994	9	A	1	0.10
1	1	Ag	2009	7	B	1	0.30
2	1	GR	1993	8	A	1	0.01
3	1	GR	1995	9	A	1	0.10
4	1	GR	1996	5	A	1	0.10

		ignitions	area_km2	rate	rate_low	rate_high
US_L3CODE	FUEL_GROUP					
1	GR	41	2602.813	0.477	0.343	0.648
	GS	28	1398.034	0.607	0.403	0.877
	SH	34	804.172	1.281	0.887	1.790
	TU	403	24546.491	0.498	0.450	0.549
	TL	450	21583.316	0.632	0.575	0.693
	Ag	2	70.921	0.855	0.103	3.087

		ign_share	area_share	rel_rate
US_L3CODE	FUEL_GROUP			
1	GR	0.043	0.051	0.839
	GS	0.029	0.027	1.066
	SH	0.035	0.016	2.251
	TU	0.421	0.481	0.874
	TL	0.470	0.423	1.110
	Ag	0.002	0.001	1.501

```
                            US_L3NAME                        NA_L1NAME  \
US_L3CODE
23          Arizona/New Mexico Mountains                Temperate Sierras
16                        Idaho Batholith  Northwestern Forested Mountains
15                      Northern Rockies  Northwestern Forested Mountains
11                        Blue Mountains  Northwestern Forested Mountains
5                          Sierra Nevada  Northwestern Forested Mountains

            ignitions  burnable_km2  rate top_fuel_group
US_L3CODE
23              33738     106791.33  9.57             TL
16              14365      55905.95  7.79             TU
15              19236      77786.70  7.49             TU
11              16975      68964.21  7.46             TU
5               10829      44759.43  7.33             TU
```

Interactive dashboard: ignition metrics by fuel group, with a choropleth / KDE map

The bar chart compares the six fuel groups. By default it shows all of CONUS; click an ecoregion on the map to see the same chart for that ecoregion alone. Hover over a bar for its counts, area, rate, and confidence interval. - **Metric** switches between the ignition rate, relative rate, shares, and raw counts (bar chart and choropleth together) - **Sort bars by value** orders the bars from highest to lowest - **The area slider** greys out fuels that cover too little area for the rate to be trustworthy (a few fires on a tiny patch of fuel give a huge but meaningless rate). It matters most for single ecoregions.

Filters (years, months, and fire size class) apply to everything at once: the bars, the choropleth, and the KDE. Rates are always per year, so a short range of years is still comparable to the full 33 years. The choropleth colors keep the scale of the full data set, so filtering to a smaller subset of fires makes the map paler instead of rescaling it.

The map has two views, chosen with the **Choropleth / KDE** switch: - **Choropleth** colors each ecoregion by the chosen metric for **one fuel group** at a time (chosen with the menu, or by clicking a bar). - **KDE** draws the kernel density of the fires that pass the filters, using the fuel groups switched on in the row of buttons (click a bar to turn that group on or off). The bandwidth slider changes the smoothing, and 5.1 km is the cross-validated value from Iteration 1. The surface is recomputed in your browser each time a control changes, from a table of all 319,148 fires saved inside the HTML file. To keep it fast, the KDE is calculated on a 5 km grid, coarser than the 1 km grid of View 2, so fine detail is slightly softer. - Hovering over an ecoregion in either view shows its counts and rates for the fuel groups shown.

Pan and zoom the map with the toolbar (useful for the small eastern ecoregions).

Error bars on the ignition rate are exact Poisson 95% confidence intervals, the same as Figure 1 (computed from a lookup table for up to 3,000 fires and a standard approximation above that, which is accurate at those counts). Relative-rate bars start at 1, the neutral value: bars to the right mean that fuel ignites more than its share of the area predicts.

Everything is also saved as a standalone HTML file in `outputs/`. It works without Jupyter, so it can be shared or linked from the PDF.

[23]:
```
output_notebook(hide_banner=True)

MIN_FUEL_KM2 = 100           # default cut-off for "too little fuel to trust the rate"
NO_FUEL = "#f0f0f0"          # fuel group doesn't exist in this ecoregion
LOW_AREA = "#bdbdbd"         # fuel exists, but covers less than the slider's area
KDE_GREY = "#e0e0e0"         # ecoregion fill underneath the KDE (the same grey as the View 5 maps)

# One row per ecoregion x fuel group, with the ecoregion's names attached
hm = eco_fuel.reset_index().join(eco_info, on="US_L3CODE")
hm["fuel_name"] = hm["FUEL_GROUP"].map(names)
hm["scope_label"] = hm["US_L3NAME"] + " (" + hm["US_L3CODE"].astype(str) + ")"
reliable = hm["area_km2"] >= MIN_FUEL_KM2
```

```
# Color scale bounds come from the 2nd-98th percentile of the trustworthy cells, so a few extreme
 ↪cells don't wash out the rest
def bounds(col):
    vals = hm.loc[reliable & (hm[col] > 0), col]
    return vals.quantile(0.02), vals.quantile(0.98)

rate_lo, rate_hi = bounds("rate")
rel_lo, rel_hi = bounds("rel_rate")
rel_max = max(1 / rel_lo, rel_hi)    # symmetric around 1 on a log scale, so 1 is the neutral middle
 ↪color
count_lo, count_hi = bounds("ignitions")

# metric -> (label, matplotlib colormap, normalization). The colormap and normalization color the
 ↪choropleth; the
# bars use the fuel group colors from the earlier maps. These scales are fixed (they come from the
 ↪full data set).
metrics = {
    "rate":       ("Ignition rate (ignitions per 1,000 km² of fuel per year)", "YlOrRd",
 ↪LogNorm(rate_lo, rate_hi)),
    "rel_rate":   ("Relative rate (share of ignitions ÷ share of fuel area)", "RdBu_r", LogNorm(1 /
 ↪rel_max, rel_max)),
    "ign_share":  ("Share of ignitions (of the area selected)", "Purples", Normalize(0, 1)),
    "area_share": ("Share of fuel area (of the area selected)", "Greens", Normalize(0, 1)),
    "ignitions":  ("Ignition count (years, months, and sizes selected)", "YlOrRd",
 ↪LogNorm(max(count_lo, 1), count_hi)),
}
# Axis number format for the bar chart, per metric
tick_formats = {"rate": "0.[0]", "rel_rate": "0.[0]", "ign_share": "0%", "area_share": "0%",
 ↪"ignitions": "0,0"}

# For each metric: the Bokeh color mapper for the color bar, plus the 256-color palette and (low,
 ↪high, is_log)
# so the browser can color the choropleth itself with the same rule as matplotlib
mappers, palettes, norm_info = {}, {}, {}
for key, (label, cmap_name, norm) in metrics.items():
    palettes[key] = [to_hex(c) for c in plt.get_cmap(cmap_name)(np.linspace(0, 1, 256))]
    is_log = isinstance(norm, LogNorm)
    norm_info[key] = [float(norm.vmin), float(norm.vmax), is_log]
    Mapper = LogColorMapper if is_log else LinearColorMapper
    mappers[key] = Mapper(palette=palettes[key], low=norm.vmin, high=norm.vmax)

# Starting colors for the choropleth (rate, fuel group 1): no fuel -> light grey, too little fuel ->
 ↪grey
rate_norm = metrics["rate"][2]
rate_rgba = plt.get_cmap("YlOrRd")(rate_norm(np.clip(hm["rate"].to_numpy(dtype=float), rate_norm.
 ↪vmin, rate_norm.vmax)))
hm["fill"] = np.where(hm["area_km2"] == 0, NO_FUEL,
                      np.where(hm["area_km2"] < MIN_FUEL_KM2, LOW_AREA, [to_hex(c) for c in rate_rgba]))

# Position of each cell's ecoregion (row in eco_info) and fuel group (in `groups`), so the browser can
# find "the cell for ecoregion j and fuel group k" as index j * len(groups) + k (hm is ecoregion-major)
hm["eco_pos"] = hm["US_L3CODE"].map({code: j for j, code in enumerate(eco_info.index)})
hm["fuel_pos"] = hm["FUEL_GROUP"].map({g: k for k, g in enumerate(groups)})
assert (hm.index == hm["eco_pos"] * len(groups) + hm["fuel_pos"]).all()

# Fixed facts about each ecoregion x fuel group (fuel area, names). Everything that depends on the
 ↪filters
# (counts, rates, shares) is recomputed in the browser from the fire table below.
```

```
source = ColumnDataSource(hm[["US_L3NAME", "scope_label", "NA_L1NAME", "fuel_name", "area_km2"]])

# --- The fire table the browser filters ---
# One row per fire with small integer codes: position on the 1 km grid, fuel group, year, month, size class, ecoregion.
# Years, months and sizes are what the filters use; the grid position is what the KDE smooths.
first_year, last_year = int(natural_ignitions["FIRE_YEAR"].min()), int(natural_ignitions["FIRE_YEAR"].max())
assert last_year - first_year + 1 == n_years
fire_size_class = fire_attrs["SIZE_CLASS"].cat.codes.to_numpy()       # 0-6 for A-G; -1 would mean a missing size
assert fire_size_class.min() >= 0
fire_table = ColumnDataSource({
    "col": natural_ignitions["COL"].to_numpy().astype("uint16"),
    "row": natural_ignitions["ROW"].to_numpy().astype("uint16"),
    "group": fire_attrs["FUEL_GROUP"].map({g: k for k, g in enumerate(groups)}).to_numpy().astype("uint8"),
    "year": (fire_attrs["FIRE_YEAR"].to_numpy() - first_year).astype("uint8"),
    "month": fire_attrs["MONTH"].to_numpy().astype("uint8"),
    "size": fire_size_class.astype("uint8"),
    "eco": fire_attrs["US_L3CODE"].map({code: j for j, code in enumerate(eco_info.index)}).to_numpy().astype("uint8"),
})

# Exact Poisson 95% limits on a count c (same method as the rate CI above), for c = 0..3,000. Above that the browser
# uses the Wilson-Hilferty approximation of the chi-square quantile, which is accurate for counts that large.
ci_counts = np.arange(3_001)
ci_lower = np.where(ci_counts > 0, chi2.ppf(0.025, 2 * ci_counts) / 2, 0).tolist()
ci_upper = (chi2.ppf(0.975, 2 * ci_counts + 2) / 2).tolist()

# --- KDE grid for the browser ---
# Fires are binned onto a 5 km grid (5 x 5 of the 1 km cells), spreading each fire over the 4 nearest cell centers
# ("linear binning") so that placing it on a coarser grid loses almost no position accuracy, then smoothed with the
# same Gaussian filter as View 2. The browser repeats this each time a control changes.
KDE_RES = 5
kde_cell = KDE_RES * cell_size
kde_ncols, kde_nrows = int(np.ceil(ncols / KDE_RES)), int(np.ceil(nrows / KDE_RES))

# CONUS mask on the coarse grid: a coarse cell is kept if any of its 1 km cells is inside CONUS
padded = np.zeros((kde_nrows * KDE_RES, kde_ncols * KDE_RES), dtype=bool)
padded[:nrows, :ncols] = conus_mask
kde_mask = padded.reshape(kde_nrows, KDE_RES, kde_ncols, KDE_RES).any(axis=(1, 3))

def kde_surface(cols, rows, bandwidth_m):
    """Ignitions per km² from fires at 1 km grid positions (cols, rows), the same steps the browser runs."""
    x, y = (cols + 0.5) / KDE_RES - 0.5, (rows + 0.5) / KDE_RES - 0.5     # position in coarse cell-center units
    x0, y0 = np.floor(x).astype(int), np.floor(y).astype(int)
    fx, fy = x - x0, y - y0
    grid = np.zeros((kde_nrows, kde_ncols))
    for dx, wx in ((0, 1 - fx), (1, fx)):
        for dy, wy in ((0, 1 - fy), (1, fy)):
            xi, yi = x0 + dx, y0 + dy
```

```
            ok = (xi >= 0) & (xi < kde_ncols) & (yi >= 0) & (yi < kde_nrows)
            np.add.at(grid, (yi[ok], xi[ok]), (wx * wy)[ok])
    smooth = gaussian_filter(grid, sigma=bandwidth_m / kde_cell, mode="constant", cval=0.0,
 ↪truncate=4.0)
    return smooth / (kde_cell / 1_000) ** 2

kde_start = np.where(kde_mask, kde_surface(natural_ignitions["COL"].to_numpy(),
 ↪natural_ignitions["ROW"].to_numpy(),
                                           bandwidth), np.nan).astype("float32")
kde_source = ColumnDataSource({"image": [kde_start], "x": [xmin], "y": [ymax],
                               "dw": [kde_ncols * kde_cell], "dh": [kde_nrows * kde_cell]})
kde_aux = ColumnDataSource({"mask": kde_mask.ravel().astype("uint8")})     # which cells are inside
 ↪CONUS

# KDE colors: the same yellow-red log scale as View 2 and View 5 (below 0.001 per km² is left
 ↪transparent)
kde_low, kde_high = 1e-3, float(np.nanmax(kde_start))
kde_mapper = LogColorMapper(palette=[to_hex(c) for c in plt.get_cmap("YlOrRd")(np.linspace(0, 1,
 ↪256))],
                            low=kde_low, high=kde_high, low_color=(0, 0, 0, 0), nan_color=(0, 0, 0, 0))

# CONUS-wide rows for the bar chart's default view: all ecoregions combined, one row per fuel group.
conus = receptivity.rename_axis("FUEL_GROUP").reset_index()[["FUEL_GROUP", "ignitions", "area_km2"]]
conus["ignitions"] = conus["ignitions"].astype(float)
conus["ign_share"] = conus["ignitions"] / conus["ignitions"].sum()
conus["area_share"] = conus["area_km2"] / conus["area_km2"].sum()
conus["rel_rate"] = conus["ign_share"] / conus["area_share"]
conus["rate"] = receptivity["rate"].to_numpy()                  # from Figure 1
conus["rate_low"], conus["rate_high"] = receptivity["rate_low"].to_numpy(), receptivity["rate_high"].
 ↪to_numpy()
conus["fuel_name"] = conus["FUEL_GROUP"].map(names)
conus["scope_label"] = "All of CONUS"
conus["NA_L1NAME"] = "–"

hover_rows = [
    ("Area", "@scope_label"),
    ("Level I region", "@NA_L1NAME"),
    ("Fuel group", "@fuel_name"),
    ("Ignitions", "@ignitions{0,0}"),
    ("Fuel area", "@area_km2{0,0} km²"),
    ("Ignition rate", "@rate{0.00} per 1,000 km²/yr (95% CI @rate_low{0.00}–@rate_high{0.00})"),
    ("Share of ignitions", "@ign_share{0.0%}"),
    ("Share of fuel area", "@area_share{0.0%}"),
    ("Relative rate", "@rel_rate{0.00}×"),
]

# --- Bar chart ---
# The bars' own source: six rows (one per fuel group) holding whichever area is selected, starting
 ↪with CONUS.
# value = the bar's length (from `base`); lo / hi = the confidence interval, drawn for the rate only.
bar_columns = ["fuel_name", "scope_label", "NA_L1NAME", "ignitions", "area_km2", "rate", "rate_low",
               "rate_high", "ign_share", "area_share", "rel_rate"]
bar_source = ColumnDataSource({
    **{col: conus[col].to_numpy() for col in bar_columns},
    "value": conus["rate"].to_numpy(), "base": np.zeros(len(groups)),
    "lo": conus["rate_low"].to_numpy(), "hi": conus["rate_high"].to_numpy(),
    "bar_fill": np.array(colors),
})
```

```
bar_x = Range1d(0, conus["rate_high"].max() * 1.08)
# Bokeh draws the first factor at the bottom, so list the groups from lowest to highest rate (highest
 ↪on top)
bar_fig = figure(y_range=FactorRange(factors=conus.sort_values("rate")["fuel_name"].tolist()),
                 x_range=bar_x, width=560, height=380, tools="tap", toolbar_location=None,
                 title="All of CONUS (all ecoregions combined)")
bar_fig.hbar(y="fuel_name", left="base", right="value", height=0.7, source=bar_source,
             fill_color="bar_fill", line_color="#555555", line_width=0.8, nonselection_fill_alpha=1,
             nonselection_line_alpha=1, selection_line_color="black", selection_line_width=3)
bar_fig.add_layout(Whisker(source=bar_source, base="fuel_name", lower="lo", upper="hi",
 ↪dimension="width",
                            level="overlay"))      # overlay: drawn in front of the bars
ref_line = Span(location=1, dimension="height", line_color="black", line_dash="dashed",
 ↪visible=False)
bar_fig.add_layout(ref_line)                       # the neutral value 1, shown for the relative rate
bar_fig.add_tools(HoverTool(tooltips=hover_rows))
bar_fig.xaxis.formatter = NumeralTickFormatter(format=tick_formats["rate"])
bar_fig.xaxis.axis_label = metrics["rate"][0]
bar_fig.ygrid.grid_line_color = None
bar_fig.yaxis.major_label_text_font_size = "10pt"
bar_fig.outline_line_color = None

# --- Map: ecoregion polygons (choropleth) with the KDE image on top of them ---
# Simplify the outlines (1 km tolerance) so the saved HTML stays small; invisible at this map size
eco_geoms = ecoregion.set_index(ecoregion["US_L3CODE"].astype(int)).geometry.simplify(1_000).
 ↪loc[eco_info.index]

def rings(geom):
    """Bokeh multi_polygons format: a list of polygons, each a list of rings (exterior first, then
 ↪holes)."""
    polys = geom.geoms if geom.geom_type == "MultiPolygon" else [geom]
    xs = [[np.round(ring.xy[0]).tolist() for ring in (poly.exterior, *poly.interiors)] for poly in
 ↪polys]
    ys = [[np.round(ring.xy[1]).tolist() for ring in (poly.exterior, *poly.interiors)] for poly in
 ↪polys]
    return xs, ys

# One row per ecoregion (same order as eco_info), starting on the first fuel group. The browser
 ↪refills these
# columns whenever a control changes.
map_df = hm[hm["FUEL_GROUP"] == groups[0]].drop(columns="FUEL_GROUP").reset_index(drop=True)
map_df["xs"], map_df["ys"] = zip(*(rings(geom) for geom in eco_geoms))
map_source = ColumnDataSource(map_df)

m = figure(width=640, height=430, match_aspect=True, tools="pan,wheel_zoom,reset",
           toolbar_location="below", title=f"Map: {names[groups[0]]}")
# Layers, bottom to top: colored ecoregion fill, KDE image (hidden until KDE is chosen), then
 ↪ecoregion outlines
# (white on the choropleth, grey over the KDE). The outlines take the clicks and hovers, and the fill
 ↪alpha of 0
# keeps them see-through. selection_line_* outlines the clicked ecoregion in black.
fill_layer = m.multi_polygons(xs="xs", ys="ys", source=map_source, fill_color="fill",
 ↪line_color=None,
                              nonselection_fill_alpha=1)
kde_layer = m.image(image="image", x="x", y="y", dw="dw", dh="dh", source=kde_source,
 ↪color_mapper=kde_mapper,
                    origin="top_left", anchor="top_left", visible=False)
outline_args = dict(xs="xs", ys="ys", source=map_source, fill_alpha=0, line_width=0.4,
```

```
                   nonselection_fill_alpha=0, nonselection_line_alpha=1,
                   selection_line_color="black", selection_line_width=2.5)
white_outlines = m.multi_polygons(line_color="white", nonselection_line_color="white",
 ↪**outline_args)
grey_outlines = m.multi_polygons(line_color="#777777", nonselection_line_color="#777777",
 ↪visible=False, **outline_args)
outline_layers = [white_outlines, grey_outlines]
m.add_tools(TapTool(renderers=outline_layers), HoverTool(tooltips=hover_rows,
 ↪renderers=outline_layers))
m.axis.visible = False
m.grid.grid_line_color = None
m.outline_line_color = None
map_bar = ColorBar(color_mapper=mappers["rate"], title=metrics["rate"][0], orientation="horizontal",
                   width=420, height=12, location=(0, 0), padding=10,
                   formatter=NumeralTickFormatter(format="0.[00]"))   # plain numbers instead of 10⁰,
 ↪10¹ on log scales
m.add_layout(map_bar, "above")

# --- Widgets ---
metric_select = Select(title="Metric", value="rate", width=380,
                       options=[(k, label) for k, (label, _, _) in metrics.items()])
sort_bars = Checkbox(label="Sort bars by value", active=True)
area_slider = Slider(title="Grey out fuels covering less than (km²)", start=0, end=2_000, step=50,
                     value=MIN_FUEL_KM2, width=380)
year_slider = RangeSlider(title="Years", start=first_year, end=last_year, value=(first_year,
 ↪last_year),
                          step=1, width=380)
month_names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
month_buttons = CheckboxButtonGroup(labels=month_names, active=list(range(12)))
size_buttons = CheckboxButtonGroup(labels=list("ABCDEFG"), active=list(range(7)))
size_note = Div(text="Fire size class (acres): A &lt; 0.25   B 0.25–9.9   C 10–99.9  
 ↪D 100–299   "
                     "E 300–999   F 1,000–4,999   G 5,000+")
fire_count = Div(text=f"<b>{len(natural_ignitions):,}</b> of {len(natural_ignitions):,} ignitions
 ↪pass the filters")
view_switch = RadioButtonGroup(labels=["Choropleth", "KDE"], active=0)
map_fuel = Select(title="Choropleth: fuel group shown", value=groups[0], width=300,
                  options=[(g, names[g]) for g in groups])
kde_buttons = CheckboxButtonGroup(labels=[names[g] for g in groups], active=list(range(len(groups))))
bandwidth_slider = Slider(title="KDE bandwidth, σ (km)", start=3, end=15, step=0.1,
                          value=round(bandwidth / 1_000, 1), width=300)
kde_scale = Select(title="KDE color scale", value="fixed", width=300,
                   options=[("fixed", "Fixed (same as View 2)"), ("auto", "Fit to the current
 ↪filters")])
grey_key = Div(text=f"<span style='background:{NO_FUEL};padding:0 8px'> </span> fuel group
 ↪absent   "
               f"<span style='background:{LOW_AREA};padding:0 8px'> </span> too little fuel area
 ↪to trust the rate")

# Runs in the browser whenever a control changes or an ecoregion is clicked on the map. It counts the
 ↪fires that
# pass the filters, then refills the bars and the choropleth, and (in KDE view) rebuilds the density
 ↪surface.
update = CustomJS(
    args=dict(fires=fire_table, source=source, map_source=map_source, bar_source=bar_source,
              kde_source=kde_source, kde_aux=kde_aux, kde_mapper=kde_mapper, kde_fixed=[kde_low,
 ↪kde_high],
              kde_dims=[kde_ncols, kde_nrows, KDE_RES, kde_cell],
```

```
            metric=metric_select, sort_bars=sort_bars, area_slider=area_slider,
↪year_slider=year_slider,
            month_buttons=month_buttons, size_buttons=size_buttons, fire_count=fire_count,
            view_switch=view_switch, map_fuel=map_fuel, kde_buttons=kde_buttons,
            bandwidth_slider=bandwidth_slider, kde_scale=kde_scale,
            map_bar=map_bar, map_fig=m, bar_fig=bar_fig, x_axis=bar_fig.xaxis[0], x_range=bar_x,
            y_range=bar_fig.y_range, ref_line=ref_line, fill_layer=fill_layer, kde_layer=kde_layer,
            white_outlines=white_outlines, grey_outlines=grey_outlines,
            mappers=mappers, palettes=palettes, norm_info=norm_info, ci_lower=ci_lower,
↪ci_upper=ci_upper,
            titles={k: label for k, (label, _, _) in metrics.items()}, tick_formats=tick_formats,
            fuel_names=names, groups=groups, group_colors=colors, first_year=first_year,
            no_fuel=NO_FUEL, low_area=LOW_AREA, kde_grey=KDE_GREY),
  code="""
  const n = groups.length;                        // fuel groups
  const NE = source.data.area_km2.length / n;     // ecoregions
  const m = metric.value;
  const kdeMode = view_switch.active == 1;

  // ---- Which fires pass the filters (years, months, size classes), counted per ecoregion x fuel
↪group ----
  const F = fires.data;
  const y0 = year_slider.value[0] - first_year, y1 = year_slider.value[1] - first_year;
  const years = y1 - y0 + 1;
  const monthOk = new Uint8Array(13);
  for (const a of month_buttons.active) monthOk[a + 1] = 1;
  const sizeOk = new Uint8Array(7);
  for (const a of size_buttons.active) sizeOk[a] = 1;
  const kdeOn = new Uint8Array(n);
  for (const a of kde_buttons.active) kdeOn[a] = 1;

  const NC = kde_dims[0], NR = kde_dims[1], RES = kde_dims[2], cellM = kde_dims[3];
  const grid = kdeMode ? new Float32Array(NC * NR) : null;
  const counts = new Float64Array(NE * n);
  let nPass = 0, nKde = 0;
  for (let f = 0; f < F.col.length; f++) {
      if (F.year[f] < y0 || F.year[f] > y1 || !monthOk[F.month[f]] || !sizeOk[F.size[f]]) continue;
      const g = F.group[f];
      counts[F.eco[f] * n + g] += 1;
      nPass++;
      if (kdeMode && kdeOn[g]) {
          // Linear binning: share the fire among the 4 nearest coarse cell centers
          nKde++;
          const x = (F.col[f] + 0.5) / RES - 0.5, y = (F.row[f] + 0.5) / RES - 0.5;
          const xi = Math.floor(x), yi = Math.floor(y), fx = x - xi, fy = y - yi;
          for (let dy = 0; dy < 2; dy++) {
              const r = yi + dy;
              if (r < 0 || r >= NR) continue;
              const wy = dy ? fy : 1 - fy;
              for (let dx = 0; dx < 2; dx++) {
                  const c = xi + dx;
                  if (c < 0 || c >= NC) continue;
                  grid[r * NC + c] += (dx ? fx : 1 - fx) * wy;
              }
          }
      }
  }
  fire_count.text = "<b>" + nPass.toLocaleString() + "</b> of " + F.col.length.toLocaleString() +
                    " ignitions pass the filters" +
```

```
                    (kdeMode ? " (" + nKde.toLocaleString() + " in the KDE's fuel groups)" : "");

   // ---- Rates and shares from counts and fuel areas ----
   const Z = 1.959964;
   // Wilson-Hilferty approximation of half a chi-square quantile (counts above the lookup table)
   const wh = (nu, z) => { const a = 2 / (9 * nu); return nu * Math.pow(1 - a + z * Math.sqrt(a), 3)
↪/ 2; };
   const ciLo = (c) => c == 0 ? 0 : (c < ci_lower.length ? ci_lower[c] : wh(2 * c, -Z));
   const ciHi = (c) => c < ci_upper.length ? ci_upper[c] : wh(2 * c + 2, Z);
   // Writes ignitions, area, rate (with 95% CI), shares and relative rate into row i of the columns
↪in o.
   // c, area: this fuel group (or set of groups); cTot, aTot: everything in the same ecoregion (or
↪in CONUS)
   const put = (o, i, c, area, cTot, aTot) => {
       o.ignitions[i] = c;
       o.area_km2[i] = area;
       if (area > 0) {
           const sc = 1000 / (area * years);
           o.rate[i] = c * sc; o.rate_low[i] = ciLo(c) * sc; o.rate_high[i] = ciHi(c) * sc;
       } else {
           o.rate[i] = NaN; o.rate_low[i] = NaN; o.rate_high[i] = NaN;
       }
       o.area_share[i] = aTot > 0 ? area / aTot : NaN;
       o.ign_share[i] = cTot > 0 ? c / cTot : NaN;
       o.rel_rate[i] = (area > 0 && cTot > 0 && aTot > 0) ? (c / cTot) / (area / aTot) : NaN;
   };
   // Color rule of the choropleth, the same as matplotlib's: clip to the scale, then pick one of
↪256 colors
   const colorOf = (mt, v) => {
       const [lo, hi, isLog] = norm_info[mt];
       v = Math.min(Math.max(v, lo), hi);
       const t = isLog ? (Math.log(v) - Math.log(lo)) / (Math.log(hi) - Math.log(lo)) : (v - lo) /
↪(hi - lo);
       const pal = palettes[mt];
       return pal[Math.min(pal.length - 1, Math.max(0, Math.floor(t * pal.length)))];
   };

   const area = source.data.area_km2;
   const eTot = new Float64Array(NE), aTotE = new Float64Array(NE);        // per ecoregion, all
↪fuel groups
   const gTot = new Float64Array(n), aTotG = new Float64Array(n);          // per fuel group, all of
↪CONUS
   for (let e = 0; e < NE; e++) {
       for (let g = 0; g < n; g++) {
           eTot[e] += counts[e * n + g]; aTotE[e] += area[e * n + g];
           gTot[g] += counts[e * n + g]; aTotG[g] += area[e * n + g];
       }
   }
   const allFires = gTot.reduce((s, v) => s + v, 0), allArea = aTotG.reduce((s, v) => s + v, 0);

   // ---- Map: each ecoregion takes the values of the fuel group shown (choropleth) or the KDE's
↪fuel groups ----
   const k = groups.indexOf(map_fuel.value);
   const kdeGroups = kde_buttons.active.map(g => groups[g]).map(g => fuel_names[g]);
   const md = map_source.data;
   for (let e = 0; e < NE; e++) {
       let c = 0, a = 0;
       for (let g = 0; g < n; g++) {
           if (kdeMode ? kdeOn[g] : g == k) { c += counts[e * n + g]; a += area[e * n + g]; }
```

```
      }
      put(md, e, c, a, eTot[e], aTotE[e]);
      md.fuel_name[e] = kdeMode ? kdeGroups.join(", ") : fuel_names[map_fuel.value];
      const v = md[m][e];
      md.fill[e] = kdeMode ? kde_grey : (a == 0 || isNaN(v) ? no_fuel : (a < area_slider.value ?
↪low_area : colorOf(m, v)));
   }
   map_source.change.emit();
   map_fig.title.text = kdeMode ? "Map: kernel density of " + kdeGroups.join(", ").toLowerCase() + "
↪ignitions"
                              : "Map: " + fuel_names[map_fuel.value];

   // ---- KDE: smooth the binned fires with a Gaussian (two 1-D passes) and show them as ignitions
↪per km² ----
   kde_layer.visible = kdeMode;
   white_outlines.visible = !kdeMode;
   grey_outlines.visible = kdeMode;
   if (kdeMode) {
      const sigma = bandwidth_slider.value * 1000 / cellM;       // in grid cells
      const r = Math.floor(4 * sigma + 0.5);                    // kernel reaches 4 sigma, like the
↪Python version
      const w = new Float32Array(2 * r + 1);
      let sw = 0;
      for (let i = -r; i <= r; i++) { w[i + r] = Math.exp(-0.5 * i * i / (sigma * sigma)); sw += w[i
↪+ r]; }
      for (let i = 0; i < w.length; i++) w[i] /= sw;
      const tmp = new Float32Array(NC * NR);
      for (let row = 0; row < NR; row++) {                      // along the rows
         const base = row * NC;
         for (let c = 0; c < NC; c++) {
            let s = 0;
            for (let q = Math.max(0, c - r); q <= Math.min(NC - 1, c + r); q++) s += w[q - c + r]
↪* grid[base + q];
            tmp[base + c] = s;
         }
      }
      const img = kde_source.data.image[0], mask = kde_aux.data.mask;
      const perKm2 = 1 / Math.pow(cellM / 1000, 2);
      let top = 0;
      for (let row = 0; row < NR; row++) {                      // down the columns
         for (let c = 0; c < NC; c++) {
            const i = row * NC + c;
            if (!mask[i]) { img[i] = NaN; continue; }
            let s = 0;
            for (let q = Math.max(0, row - r); q <= Math.min(NR - 1, row + r); q++) s += w[q - row
↪+ r] * tmp[q * NC + c];
            img[i] = s * perKm2;
            if (img[i] > top) top = img[i];
         }
      }
      kde_source.change.emit();
      if (kde_scale.value == "auto" && top > 0) { kde_mapper.high = top; kde_mapper.low = top /
↪1000; }
      else { kde_mapper.low = kde_fixed[0]; kde_mapper.high = kde_fixed[1]; }
   }
   map_bar.color_mapper = kdeMode ? kde_mapper : mappers[m];
   map_bar.title = kdeMode ? "Natural ignitions per km² (years, months, and sizes selected)" :
↪titles[m];
   map_bar.formatter.format = kdeMode ? "0.[00000]" : "0.[00]";
```

```
    // ---- Bars: one row per fuel group, from the clicked ecoregion or from the CONUS totals ----
    const sel = map_source.selected.indices;
    const j = sel.length > 0 ? sel[0] : -1;
    const bd = bar_source.data;
    for (let g = 0; g < n; g++) {
        if (j < 0) put(bd, g, gTot[g], aTotG[g], allFires, allArea);
        else put(bd, g, counts[j * n + g], area[j * n + g], eTot[j], aTotE[j]);
        bd.scope_label[g] = j < 0 ? "All of CONUS" : source.data.scope_label[j * n];
        bd.NA_L1NAME[g] = j < 0 ? "-" : source.data.NA_L1NAME[j * n];
        const a = bd.area_km2[g];
        bd.value[g] = a == 0 ? NaN : bd[m][g];
        bd.base[g] = m == "rel_rate" ? 1 : 0;
        bd.lo[g] = (m == "rate" && a > 0) ? bd.rate_low[g] : NaN;     // error bars only on the rate
        bd.hi[g] = (m == "rate" && a > 0) ? bd.rate_high[g] : NaN;
        bd.bar_fill[g] = a == 0 ? no_fuel : (a < area_slider.value ? low_area : group_colors[g]);
    }
    bar_source.change.emit();

    // The outlined bars are the fuel group on the choropleth, or the fuel groups switched on in the KDE
    const want = kdeMode ? kde_buttons.active.slice().sort((a, b) => a - b) : [k];
    const cur = bar_source.selected.indices;
    if (cur.length != want.length || !want.every((v, i) => v === cur[i])) bar_source.selected.indices = want;

    // Bar order: highest value on top when sorting, otherwise the usual fuel group order
    const order = Array.from({length: n}, (_, g) => g);
    if (sort_bars.active) {
        const val = (g) => isNaN(bd.value[g]) ? -1e300 : bd.value[g];
        order.sort((a, b) => val(b) - val(a));
    }
    y_range.factors = order.map(g => fuel_names[groups[g]]).reverse();

    // Axis: start at 0, end just past the longest bar (or error bar); relative rate also shows the line at 1
    let top = 0;
    for (let g = 0; g < n; g++) {
        const t = isNaN(bd.hi[g]) ? bd.value[g] : bd.hi[g];
        if (!isNaN(t) && t > top) top = t;
    }
    if (m == "rel_rate") top = Math.max(top, 1.5);
    x_range.start = 0;
    x_range.end = top > 0 ? top * 1.08 : 1;
    ref_line.visible = (m == "rel_rate");
    x_axis.axis_label = titles[m];
    // Tick labels need more decimals when a filter leaves only small values
    let fmt = tick_formats[m];
    if (m == "rate" || m == "rel_rate") fmt = top < 0.2 ? "0.[000]" : (top < 2 ? "0.[00]" : "0.[0]");
    else if (m == "ign_share" || m == "area_share") fmt = top < 0.1 ? "0.[0]%" : "0%";
    x_axis.formatter.format = fmt;
    bar_fig.title.text = j < 0 ? "All of CONUS (all ecoregions combined)" : bd.scope_label[0];
""")
for widget in (metric_select, area_slider, map_fuel, kde_scale):
    widget.js_on_change("value", update)
for widget in (year_slider, bandwidth_slider):
    widget.js_on_change("value_throttled", update)       # recompute when the handle is released, not on every pixel
for widget in (month_buttons, size_buttons, kde_buttons, view_switch):
```

```
    widget.js_on_change("active", update)
sort_bars.js_on_change("active", update)
map_source.selected.js_on_change("indices", update)    # clicking an ecoregion (or empty space)
 ↪changes the bars

# Bar click. Choropleth view: show that bar's fuel group on the map. KDE view: switch that fuel group
 ↪on or off
# in the KDE (at least one stays on). Clicking empty space leaves things as they are.
bar_click = CustomJS(args=dict(bar_source=bar_source, map_fuel=map_fuel, kde_buttons=kde_buttons,
                               view_switch=view_switch, groups=groups), code="""
    const sel = bar_source.selected.indices;
    const same = (a, b) => a.length == b.length && a.every((v, i) => v === b[i]);
    if (view_switch.active == 1) {
        const on = kde_buttons.active.slice().sort((a, b) => a - b);
        if (same(sel, on)) return;                                  // the dashboard's own highlighting,
 ↪not a click
        if (sel.length == 1) {
            const g = sel[0];
           const next = on.includes(g) ? on.filter(x => x != g) : on.concat([g]).sort((a, b) => a - b);
            if (next.length > 0) { kde_buttons.active = next; return; }    // this triggers the update
        }
        bar_source.selected.indices = on;
    } else {
        if (sel.length == 0) { bar_source.selected.indices = [groups.indexOf(map_fuel.value)]; return;
 ↪ }
        if (map_fuel.value != groups[sel[0]]) map_fuel.value = groups[sel[0]];
    }
""")
bar_source.selected.js_on_change("indices", bar_click)
bar_source.selected.indices = [0]                       # start with the first fuel group selected,
 ↪like the map

filters = column(Div(text="<b>Filters</b> - apply to the bars, the choropleth, and the KDE"),
                 row(year_slider, column(Div(text="Months"), month_buttons)),
                 row(column(Div(text="Fire size class"), size_buttons), size_note), fire_count)
map_controls = column(view_switch, map_fuel, Div(text="KDE fuel groups (or click the bars)"),
 ↪kde_buttons,
                      row(bandwidth_slider, kde_scale))
dashboard = column(filters,
                   row(column(row(metric_select, sort_bars), area_slider, grey_key, bar_fig),
                       column(map_controls, m)))
show(dashboard)

OUTPUT_DIR.mkdir(exist_ok=True)
save(dashboard, filename=OUTPUT_DIR / "ecoregion_fuel_dashboard.html",
     title="Natural ignitions by fuel group and ecoregion")
```

```
[Interactive Bokeh dashboard - not shown in the PDF. Open
outputs/ecoregion_fuel_dashboard.html in a browser to use it.]
```

Conclusion

Spatial analysis of the raw ignition distribution and the by-fuel-type distribution shows that natural ignitions are highly concentrated, with hotspots in the forested regions of the Western US and in the Southern Coastal Plain. Six of the top seven ecoregions lie in the same Western Cordillera Level II ecoregion, which is characterized by rugged mountain ranges, mostly mixed-coniferous forest ecosystems, and climates with winter snow and warm, dry summers that experience convective thunderstorms bringing lightning. The Arizona/New Mexico Mountains are alone at the top, about 23% above the next highest ecoregion. Interestingly, the ignitions in this region are different from the other top ecoregions in the West - they are

dominated by the timber-litter and grass fuel groups, not timber-understory like the other ecoregions. This points to driving factors other than fuel type, like intense summer monsoonal lightning. It also suggests that it might not be appropriate to compare fuel types across ecoregions, but rather within ecoregions, which points out the importance of scale within the analysis.

Hotspots cluster with their neighbors - the spatial lag and Moran's I ($I = 0.616$ on the log rate, $p = 0.001$) analysis shows this, but I doubt the driving force behind it is fuel characteristics. Ecoregions show drastically different fuel types - while grass and timber can both burn, it doesn't mean these two are somehow related. It's fair to say the clustering here is most likely driven by climate and regional weather patterns.

The Southeast is a separate hotspot that warrants a closer look - particularly the Southern Coastal Plain and Southern Florida Coastal Plain. These regions are flat but experience a high degree of lightning strikes - the Southern Coastal Plain's ignitions are mostly timber fuels (timber litter 34%, timber-understory 32%), while the Southern Florida Coastal Plain's are mostly grass (54%), dominated by high-load humid-climate grasses like sawgrass. The two Southeast hotspots have different dominant fuel types and both have high ignition densities, which points to a driver other than fuel type, like lightning and climate.

Fuel type is only part of the pattern we can draw conclusions from in this spatial analysis. While timber-understory has the highest ignition rate nationally and dominates ignitions in hotspots like the Northern Rockies (48%) and Sierra Nevada (43%), the Arizona/New Mexico Mountains and the Southeast show that other driving factors like lightning frequency and regional climate shape where ignition density falls.

At the landscape scale, ignitions appear to concentrate where receptive fuels overlap with frequent lightning, but the nuances within that overlap still need to be explored with further analysis and additional datasets (lightning) before drawing definitive conclusions.

Future Work

Grouping the 40 Scott and Burgan fuel models into broad fuel groups made the patterns easier to see, but it also hides differences between individual fuel classes. For example, a low-load dry-climate grass (GR1) and a high-load humid-climate grass (GR8) may respond to ignition very differently. Future work will compare ignition rates for the individual fuel classes within each group to find trends that the grouped analysis misses.

Lightning exposure is the other major gap. The ignition rates in this analysis don't account for where lightning actually strikes, so a fuel may look receptive simply because it grows in storm-prone regions. Adding the GOES Geostationary Lightning Mapper (GLM) thunder-hour data would let us map lightning activity across CONUS and model ignitions as a function of both fuel and lightning. That would answer our original research question: whether some fuels convert lightning into ignitions more readily than others, independent of how much lightning they receive.

AI Appendix

Prompts:

1. I have three geospatial datasets: Spatial wildfire occurrence data for the United States, 1992–2020 (7th Edition) (USFS); LANDFIRE's 40 Scott and Burgan Fire Behavior Fuel Models (FBFM40) CONUS raster (USFS & USGS); and Level III Ecoregions of the Conterminous United States (EPA). Give me guidance on how to analyze these datasets to answer my research question: how are natural wildfire ignitions distributed across ecoregions, and which fuel types have the highest ignition rates relative to the area they cover?
2. Guide me through creating a grid for a KD-Trees kernel density algorithm using fuel type from the fuelClass dataset (LANDFIRE), so each grid cell contains the majority fuel class.
3. Give me the common libraries geospatial data scientists use to implement KDEs. I am going to implement a KDE on my fire-point-by-fuel-type dataset using a Gaussian function.
4. What is a more feasible computational approach: keep the same cell size and count per point using a smaller bandwidth, or drop the cell size and sum the counts in each cell? Explain the tradeoffs of these two approaches.
5. Now that I have my grid constructed and have summed the number of fires per grid cell, help me implement the suggested KDE. Walk through each step so I understand the code and the arguments being passed into the KDE function.

6. Give me the typical libraries geospatial data scientists use to rasterize KDEs. Tell me about the methods and what to take into account when rasterizing point density data.

Reflection: These prompts were pivotal in implementing my KDE, and they even pointed me toward using cross-validation (with sklearn) to find the optimal bandwidth for my Gaussian kernel. I found that chatting with GenAI, while taking the time to comment the code and ask about lines I didn't understand, helped me get further than I otherwise would have. I love seeing which libraries exist for what I'm trying to implement and getting a breakdown of each one, so I can find the one that best suits my needs. Learning to cache data on disk and to use `engine="pyogrio"` to read in large datasets was also extremely helpful for speeding up this notebook, as was figuring out the most feasible computational approach to rasterizing KDE data.